

Java 26 — Prepare to Make Final Mean Final

Java Version: 26

Year: 2026

Feature: Prepare to Make Final Mean Final

UniqueClassName: FinalMeansFinal

JEP 8346 — Prepare to Make Final Mean Final is a Java 26 preview feature. Its goal is to make the Java language and JVM consistently enforce the meaning of final: once a final field has been initialized, its value should not subsequently be changed through reflective or other privileged mechanisms.

Definition

Historically, Java's final fields have not always been absolutely immutable at runtime. Certain reflection APIs and other deep/privileged mechanisms could modify final fields after construction.

Java 26 begins the transition toward making **final mean truly final**.

The feature:

- strengthens the integrity guarantees of final fields;
- prepares the platform for eventually disallowing mutation of final fields;
- introduces a controlled mechanism for code that genuinely needs exceptional access;
- helps the JVM optimize based on the assumption that final fields remain unchanged;
- produces warnings when applications perform final-field mutation through mechanisms that will become restricted.

The important distinction is:

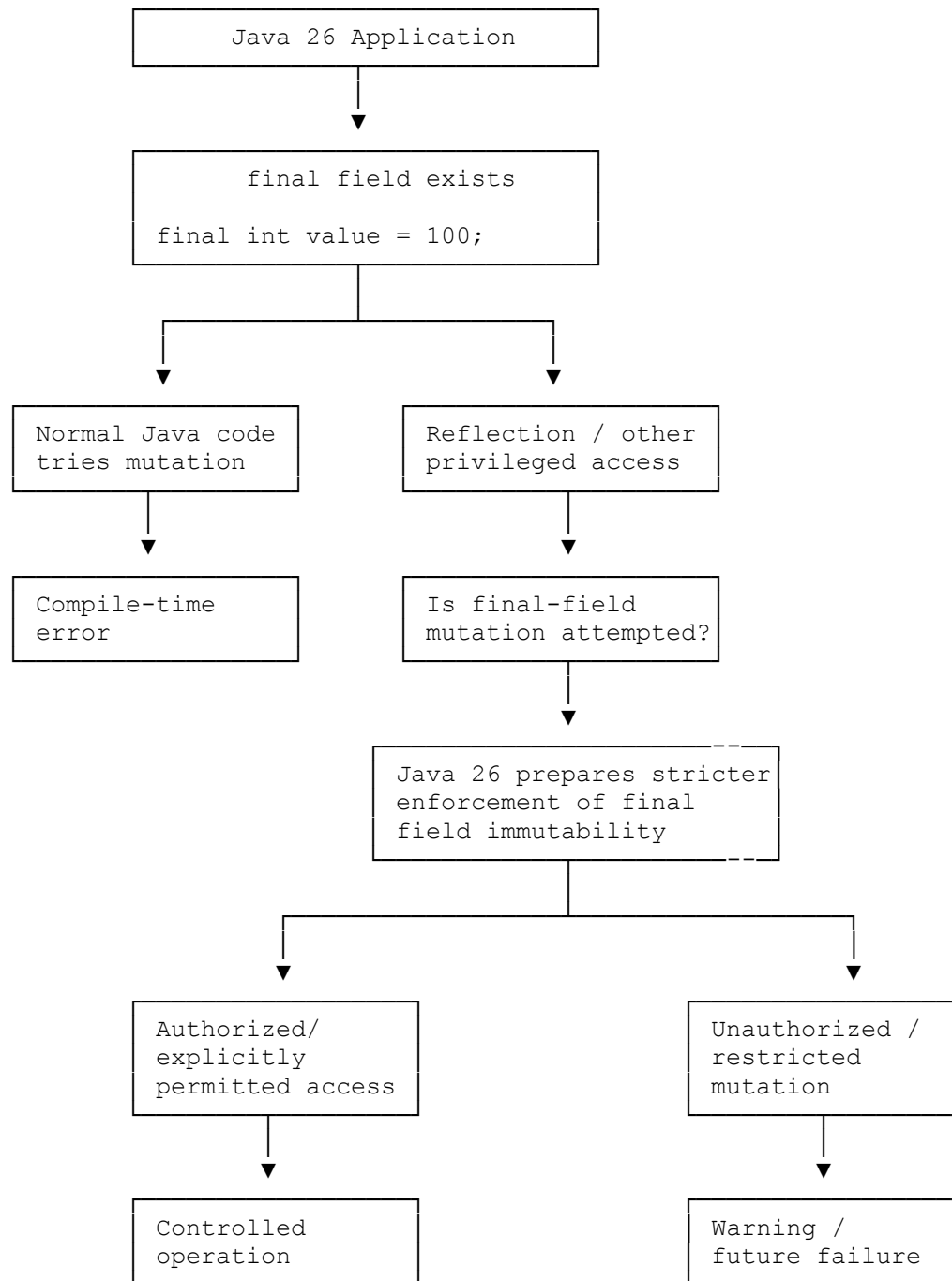
```
final int age = 25;
```

means that normal Java code cannot assign another value:

```
age = 30; // Compile-time error
```

The Java 26 change is primarily about **runtime/privileged mechanisms that previously could circumvent this rule**.

Flowchart



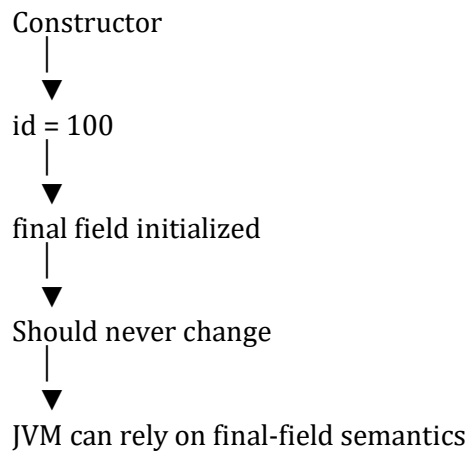
Why does this matter?

The JVM can make stronger assumptions when it knows that a final field **really cannot change** after initialization.

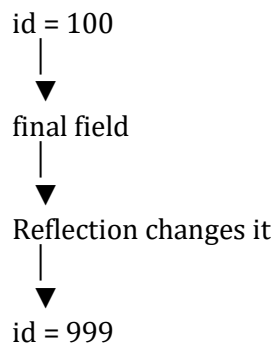
For example:

```
public final class User {  
    private final int id;  
  
    public User(int id) {  
        this.id = id;  
    }  
  
    public int getId() {  
        return id;  
    }  
}
```

Conceptually:

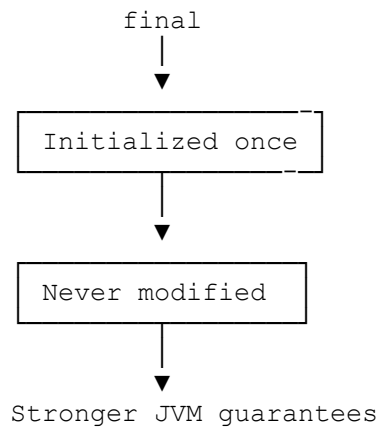


If privileged mechanisms can secretly change:



then the JVM has weaker guarantees.

Java 26 therefore starts moving the platform toward:



Java Code Example

1. Normal use of final

```
public class FinalMeansFinal {  
  
    private final int id;  
  
    public FinalMeansFinal(int id) {  
        this.id = id;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public static void main(String[] args) {  
  
        FinalMeansFinal user = new FinalMeansFinal(100);  
  
        System.out.println("ID = " + user.getId());  
  
        // user.id = 200;  
        // Compile-time error:  
        // cannot assign a value to final variable id  
    }  
}
```

Output

ID = 100

The field is initialized in the constructor:

```
this.id = id;
```

After initialization, ordinary Java code cannot assign to it again.

Example 2 — final field and reflection

Historically, privileged reflection could be used in situations where developers expected a final field to be immutable but runtime mechanisms could circumvent that expectation.

Consider:

```
import java.lang.reflect.Field;

public class FinalMeansFinal {

    private final String name;

    public FinalMeansFinal(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public static void main(String[] args) throws Exception {

        FinalMeansFinal object =
            new FinalMeansFinal("Anurag");

        System.out.println("Before = " + object.getName());

        Field field =
            FinalMeansFinal.class.getDeclaredField("name");

        field.setAccessible(true);

        // Historically, privileged reflective mechanisms
        // could be used to attempt mutation of final fields.
        //
        // field.set(object, "Java");

        System.out.println("After = " + object.getName());
    }
}
```

The important point for **Java 26** is not that developers should use reflection to change final fields. Quite the opposite: Java 26 is preparing the platform so that such mutation becomes increasingly restricted.

Example 3 — Record with final state

Records are a good illustration of why strong final-field semantics are useful.

```
public record Customer(  
    int id,  
    String name  
) {  
}
```

Conceptually, the generated state behaves like:

```
private final int id;  
private final String name;
```

Usage:

```
public class FinalMeansFinal {  
  
    public static void main(String[] args) {  
  
        Customer customer =  
            new Customer(101, "Anurag");  
  
        System.out.println(customer.id());  
        System.out.println(customer.name());  
  
        // customer.id = 200;  
        // Not allowed.  
    }  
  
    record Customer(int id, String name) {  
    }  
}
```

Output:

```
101  
Anurag
```

Key Concept

The feature can be summarized as:

Before

final

- Normal Java assignment: prohibited
- Some privileged mechanisms:
could circumvent final-field semantics

Java 26

final

- Normal assignment: prohibited
- Privileged mutation: increasingly restricted
- Warnings prepare developers for
stronger enforcement

Future direction

final

↓

Initialized once

↓

Cannot be changed

↓

Strong final-field integrity

Benefits

Benefit	Explanation
Stronger immutability	Makes final more faithfully represent an immutable field after initialization
Better JVM assumptions	The JVM can rely more confidently on final-field semantics
Safer reflection	Reduces surprising mutations through privileged mechanisms
Better encapsulation	Prevents code from bypassing intended object invariants

Future compatibility	Prepares applications for stronger final-field enforcement
Improved optimization potential	Stable final fields provide stronger information to runtime optimizations

One-line definition

Java 26's "Prepare to Make Final Mean Final" prepares the Java platform to strongly enforce that final fields, once initialized, cannot subsequently be mutated through reflective or other privileged mechanisms, improving integrity, encapsulation, and JVM optimization opportunities.

Java 26 — Remove the Applet API

Java Version: 26

Year: 2026

Feature: Remove the Applet API

UniqueClassName: RemoveAppletAPI

This feature is associated with **JEP 504 — Remove the Applet API** in Java 26. It completes the long-running removal of the obsolete Java Applet technology.

Definition

The **Applet API** was a collection of Java APIs that allowed Java programs called **applets** to be embedded in web pages and executed inside a browser using a Java browser plug-in.

Applet technology became obsolete because modern browsers removed support for Java plug-ins, and applets had already been deprecated for removal in earlier Java releases.

With **Java 26**, the Applet API is removed from the JDK.

In simple terms:

Old Java



HTML Web Page



Java Browser Plug-in



Java Applet



Applet API

Java 26 moves to:

Modern Java



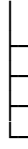
No Applet API



No Java browser plug-in model



Use modern web technologies



JavaScript / TypeScript

HTML / CSS

WebAssembly

Java backend applications

What Was the Applet API?

A traditional applet extended:

`java.applet.Applet`

For example:

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {

    @Override
    public void paint(Graphics g) {
        g.drawString("Hello, Java Applet!", 50, 50);
    }
}
```

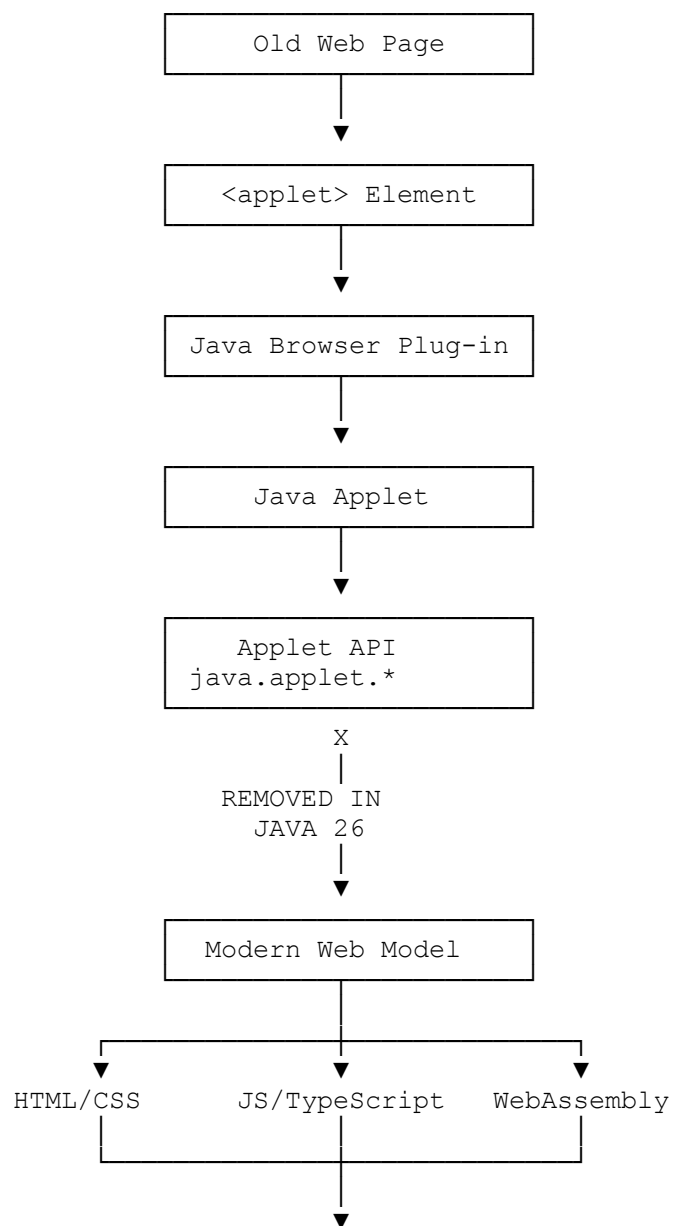
The applet could historically be embedded in an HTML page.

```
<applet code="HelloApplet.class"
  width="400"
  height="200">
</applet>
```

The browser's Java plug-in would load and execute the applet.

That architecture is now obsolete.

Flowchart



APIs Removed

The removal concerns the old `java.applet` API and related applet-specific functionality.

The important classes/interfaces historically included:

API	Purpose
<code>java.applet.Applet</code>	Base class for Java applets
<code>java.applet.AppletContext</code>	Communication with the applet's environment
<code>java.applet.AppletStub</code>	Provides applet execution context
<code>java.applet.AudioClip</code>	Simple audio playback
<code>javax.swing.JApplet</code>	Swing-based applet

The applet deployment mechanism and associated support are also obsolete.

Java Code Example — Old Applet

A typical pre-modern Java application might have contained:

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {

    @Override
    public void init() {
        System.out.println("Applet initialized");
    }

    @Override
    public void start() {
        System.out.println("Applet started");
    }

    @Override
    public void paint(Graphics g) {
```

```

        g.drawString("Hello from Java Applet!", 50, 50);
    }

    @Override
    public void stop() {
        System.out.println("Applet stopped");
    }

    @Override
    public void destroy() {
        System.out.println("Applet destroyed");
    }
}

```

This is **legacy code** and is not suitable for Java 26.

Java 26 Replacement

If the goal is simply to create a graphical Java application, use a normal application entry point instead of an applet.

```

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.SwingConstants;

public class RemoveAppletAPI {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Java 26 Application");

        JLabel label = new JLabel(
            "Hello from Java 26!",
            SwingConstants.CENTER
        );

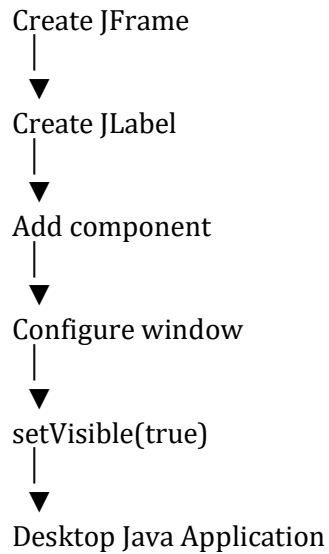
        frame.add(label);

        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}

```

Flow

main()
↓
▼



Another Replacement — Console Application

If no graphical interface is required, a simple Java application is enough:

```
public class RemoveAppletAPI {  
    public static void main(String[] args) {  
        System.out.println(  
            "Java Applets are no longer part of Java 26."  
        );  
        System.out.println(  
            "Use a modern Java application instead."  
        );  
    }  
}
```

Output

Java Applets are no longer part of Java 26.
Use a modern Java application instead.

Applet vs Modern Java Application

Aspect	Old Applet	Java 26 Application
Entry point	init(), start() etc.	main()

Execution	Browser plug-in	JVM/application launcher
API	java.applet.*	Standard Java APIs
Browser dependency	Yes	No
<applet> HTML tag	Used historically	Not used
Security model	Applet sandbox	Modern application/module/security mechanisms
Browser support	Obsolete	Not applicable
Java 26	Removed	Supported

Migration Example

Before

```
public class MyApplet extends Applet {

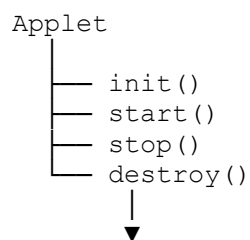
    @Override
    public void paint(Graphics g) {
        g.drawString("Hello", 20, 20);
    }
}
```

After

```
public class RemoveAppletAPI {

    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

The fundamental migration is:



```
main()
```

Why Was It Removed?

The Applet API had effectively reached the end of its useful life:

1. **Browsers stopped supporting Java plug-ins.**
2. **The <applet> deployment model became obsolete.**
3. Applets had been **deprecated for removal** before Java 26.
4. Maintaining the API added obsolete platform surface area.
5. Modern web applications use technologies such as JavaScript, HTML, CSS, and WebAssembly instead.
6. Java remains appropriate for **desktop, server, cloud, and other non-browser application architectures.**

One-line definition

Java 26 removes the obsolete java.applet API and its applet-specific support, completing the removal of Java's browser-embedded applet technology and encouraging migration to modern web technologies or ordinary Java applications.

Java 26 — Ahead-of-Time Object Caching with Any GC

Java Version: 26

Year: 2026

Feature: Ahead-of-Time Object Caching with Any GC

UniqueClassName: AOTObjectCachingAnyGC

JEP: 516

JEP 516 enhances Java's Ahead-of-Time (AOT) cache so that **cached Java heap objects can be used with any garbage collector**, including **ZGC**. It was delivered in JDK 26. (bugs.openjdk.org)

Definition

Ahead-of-Time Object Caching with Any GC allows the JVM to cache Java objects ahead of application execution and subsequently load those objects regardless of which garbage collector is being used.

The main objective is to improve **startup and warm-up performance** while removing the previous dependency between cached heap-object representation and a particular GC implementation. (bugs.openjdk.org)

Earlier AOT object caching stored cached objects in a format closely tied to the GC's object/reference representation. This created compatibility problems because different GCs could use different object layouts and reference mechanisms. In particular, advanced AOT cache functionality involving heap objects was previously unavailable with ZGC. ([OpenJDK Mail](#))

Java 26 introduces a more **GC-agnostic object-archiving/loading mechanism**.

Simple Definition

JEP 516 enables Java's AOT cache to store and restore Java heap objects in a garbage-collector-independent way, allowing AOT object caching to work with any GC, including ZGC, and thereby improving application startup and warm-up performance. (bugs.openjdk.org)

What Is AOT Object Caching?

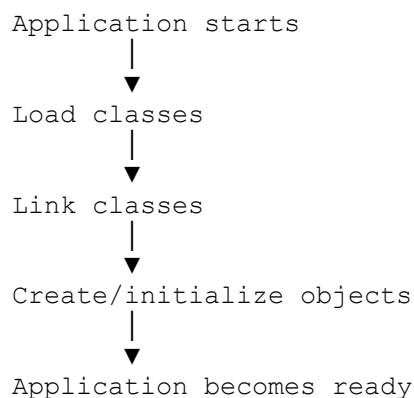
AOT means:

Ahead Of Time

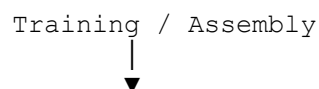
Instead of making the JVM perform every initialization task from scratch every time an application starts, Java can perform selected work **before production execution** and store the results in an **AOT cache**.

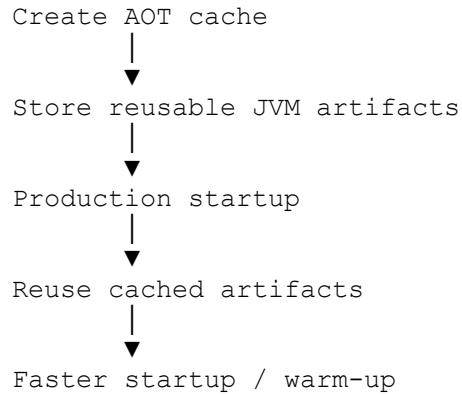
Conceptually:

Without AOT



With AOT caching:

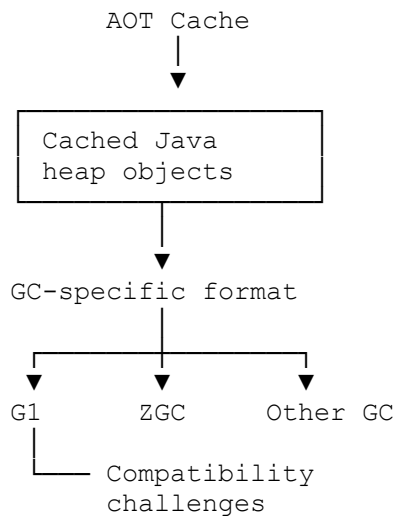




The AOT cache can contain Java classes and heap objects; the broader AOT infrastructure can also store other optimization artifacts. ([OpenJDK](#))

The Problem Before JEP 516

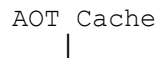
A simplified representation of the old situation is:

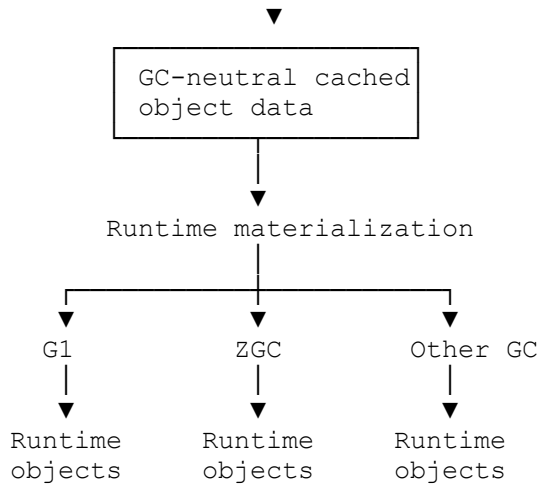


The problem was that cached heap objects could depend on the representation expected by the GC.

Java 26 Solution

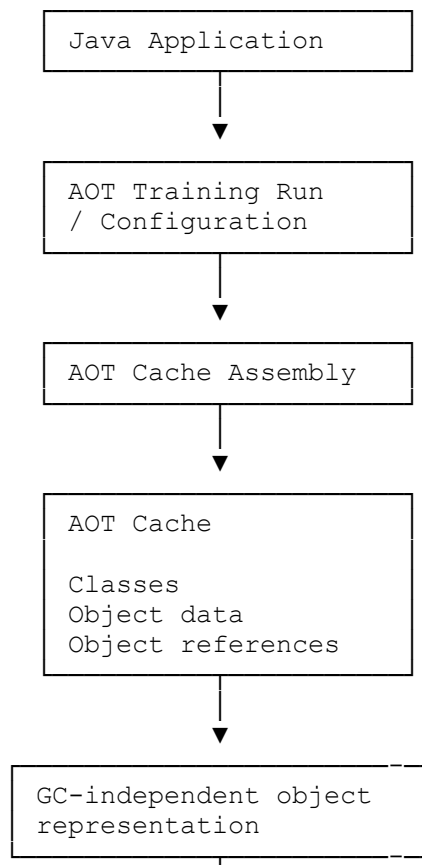
JEP 516 introduces a GC-agnostic approach.

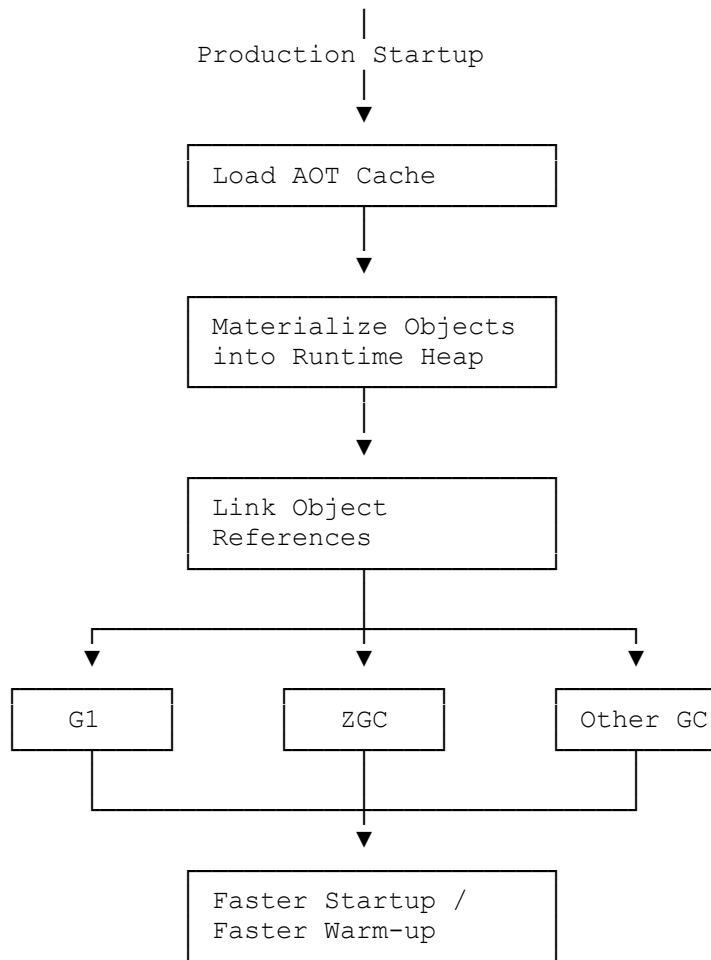




The implementation represents references between archived objects using **object indices rather than raw pointers**. At runtime, objects are materialized and their references are linked according to the requirements of the active GC. ([OpenJDK Mail](#))

Flowchart

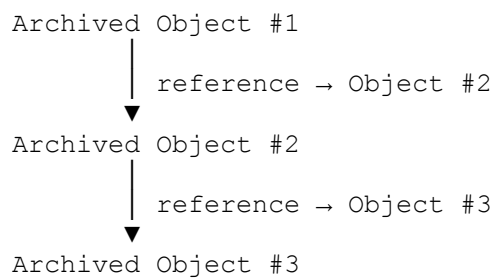




Important Internal Concept: Object Indices

One of the important ideas in JEP 516 is that references between archived objects can be represented using **object indices** instead of assuming that a raw memory pointer will be valid for every GC. ([OpenDK Mail](#))

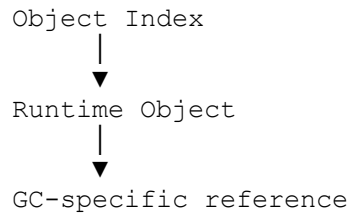
For example:



The cache can conceptually contain:

Object Index	Object
1	String
2	byte[]
3	Some Java object

At runtime:



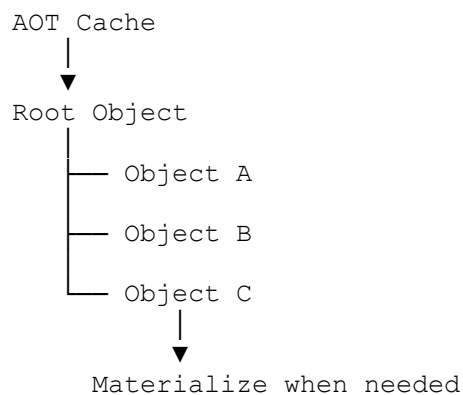
This decouples the archived representation from the particular GC being used at runtime.

Lazy Materialization

JEP 516 also supports **lazy materialization**.

That means cached objects do not necessarily have to be materialized into the runtime heap all at once.

Conceptually:



The implementation also uses concurrency and an AOTThread to perform some of the work in the background, reducing the startup impact on the application's bootstrapping thread. ([OpenJDK Mail](#))

Java Code Example

This feature is primarily a **JVM/AOT-cache feature**, not a new Java language construct. Therefore, there is no new Java keyword or class that you directly program against.

A normal Java program can benefit from it.

```
public class AOTObjectCachingAnyGC {  
  
    private static final String APPLICATION_NAME =  
        "AOT Object Caching Demo";  
  
    private static final String[] FEATURES = {  
        "AOT Class Linking",  
        "AOT Object Caching",  
        "GC-independent Object Loading"  
    };  
  
    public static void main(String[] args) {  
  
        System.out.println(APPLICATION_NAME);  
  
        for (String feature : FEATURES) {  
            System.out.println(feature);  
        }  
  
        System.out.println("Application started.");  
    }  
}
```

The application code itself does not need to contain special AOT-cache APIs.

The JVM handles the AOT caching mechanism.

AOT Cache Workflow

The general AOT workflow consists of three phases:

1. Training
↓
2. Assembly
↓
3. Production

OpenJDK describes the AOT cache workflow as **training → assembly → production**. ([OpenJDK](#))

1. Training

Run the application with a representative workload and collect information for the AOT configuration.

Conceptually:

```
java \
  -XX:AOTConfiguration=app.aotconfig \
  -cp app.jar \
  AOTObjectCachingAnyGC
```

2. Assembly

Use the generated configuration to create the AOT cache:

```
java \
  -XX:AOTMode=create \
  -XX:AOTConfiguration=app.aotconfig \
  -XX:AOTCacheOutput=app.aot \
  -cp app.jar \
  AOTObjectCachingAnyGC
```

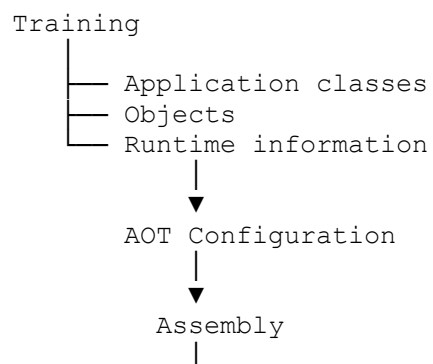
The exact production/training setup depends on the JDK build and deployment configuration. The AOT cache is tied to the application, JDK release, operating system, and CPU architecture. ([OpenJDK](#))

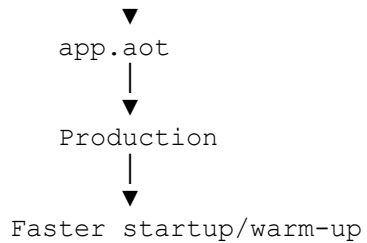
3. Production

Run the application using the generated cache:

```
java \
  -XX:AOTCache=app.aot \
  -cp app.jar \
  AOTObjectCachingAnyGC
```

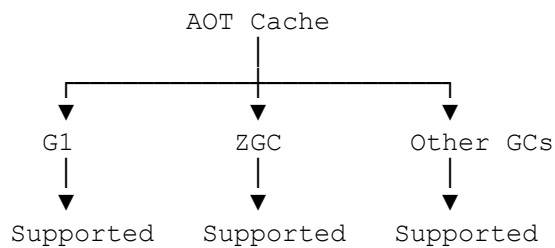
Conceptually:





Any GC

The important improvement is that the cached object mechanism is designed to work with different garbage collectors.



In particular, JEP 516 specifically addresses the previous limitation around **ZGC** and enables the more advanced AOT cache functionality for ZGC as well. (bugs.openjdk.org)

Before vs Java 26

Aspect	Earlier AOT Object Caching	Java 26 / JEP 516
Cached classes	Supported	Supported
Cached heap objects	Supported with restrictions	Enhanced
GC dependency	More tightly coupled	GC-agnostic mechanism
G1	Supported	Supported
ZGC	Object-cache limitations	Supported
Object references	GC-dependent representation	Object-index-based approach
Materialization	Limited by representation	Runtime materialization

Startup optimization	Yes	Improved applicability
Java source changes	Generally unnecessary	Generally unnecessary

Benefits

1. Faster startup

Precomputed artifacts can reduce work that the JVM needs to perform during startup.

2. Faster warm-up

The JVM can reuse information and objects prepared ahead of production execution.

3. GC independence

AOT object caching is no longer fundamentally tied to a particular GC representation.

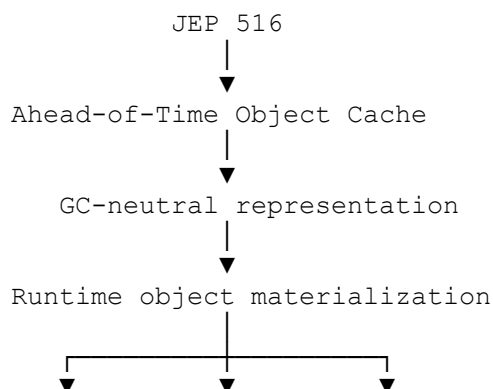
4. ZGC support

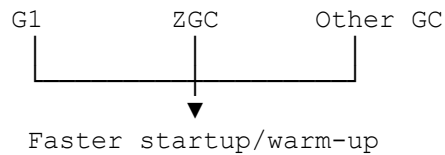
This is particularly important for applications using the low-latency ZGC.

5. Better JDK-wide caching

The GC-neutral approach makes it possible to provide baseline AOT caches that can work across different environments. (bugs.openjdk.org)

Key Takeaway





One-line definition

JEP 516 in Java 26 makes AOT object caching garbage-collector agnostic by storing archived objects in a neutral form and materializing/linking them at runtime, enabling AOT object-cache optimizations with any GC, including ZGC. (bugs.openjdk.org)

Java 26 — HTTP/3 for the HTTP Client API

Java Version: 26

Year: 2026

Feature: HTTP/3 for the HTTP Client API

UniqueClassName: HTTP3ClientAPI

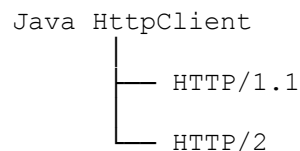
JEP: 517 — HTTP/3 for the HTTP Client API

Definition

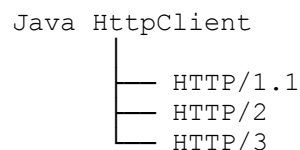
JEP 517 enhances Java's standard `java.net.http.HttpClient` API to support **HTTP/3**, allowing Java applications to communicate with HTTP/3 servers using the existing HTTP Client programming model.

HTTP/3 is the newest major version of HTTP and runs over **QUIC**, which itself uses **UDP**, rather than TCP.

Before this feature:



With Java 26:





The important point is that developers continue using `HttpClient`, `HttpRequest`, and `HttpResponse`; HTTP/3 support is integrated into the existing API rather than requiring an entirely new HTTP client API.

What Is HTTP/3?

HTTP/3 is an HTTP protocol version designed to improve web communication, particularly around latency and connection behavior.

Its protocol stack is approximately:

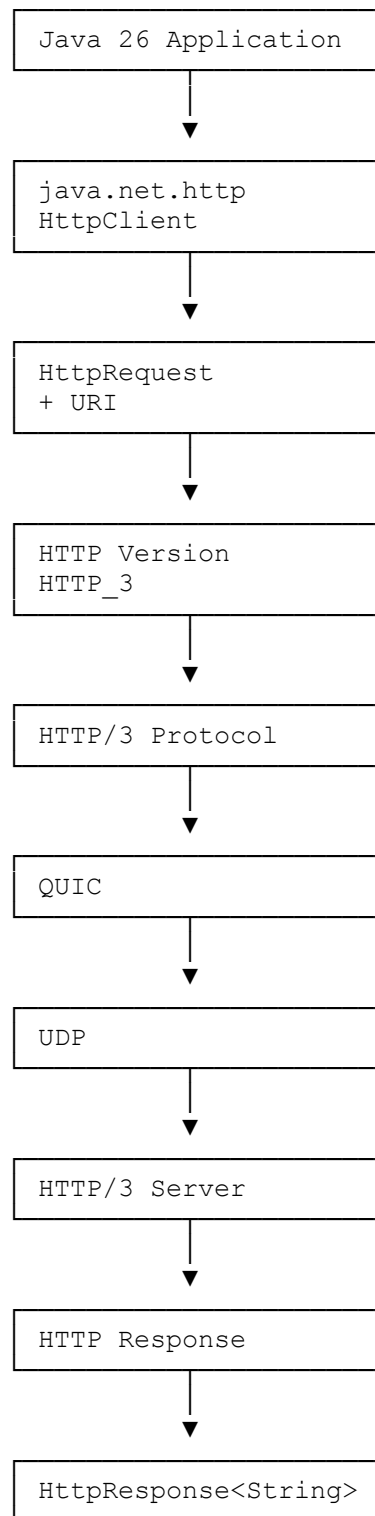
Application HTTP/3
QUIC
UDP
IP

Compare that with HTTP/2:

HTTP/2
TLS
TCP
IP

HTTP/3 uses QUIC's stream-based transport, which helps avoid some of the head-of-line blocking characteristics associated with TCP-based HTTP/2.

Flowchart



Java Code Example

The Java 26 API provides an HTTP version constant for HTTP/3:

`HttpClient.Version.HTTP_3`

A basic example:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class HTTP3ClientAPI {

    public static void main(String[] args) throws Exception {

        HttpClient client = HttpClient.newBuilder()
            .version(HttpClient.Version.HTTP_3)
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://example.com"))
            .GET()
            .build();

        HttpResponse<String> response =
            client.send(
                request,
                HttpResponse.BodyHandlers.ofString()
            );

        System.out.println("Status: "
            + response.statusCode());

        System.out.println("HTTP Version: "
            + response.version());

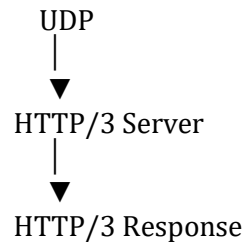
        System.out.println("Body:");
        System.out.println(response.body());
    }
}
```

Expected behavior

If the server and network support HTTP/3, the client can use HTTP/3:

HTTP/3 request





The exact negotiated protocol depends on the server and network environment.

HTTP_3 Version Selection

The key line is:

```
.version(HttpClient.Version.HTTP_3)
```

For example:

```
HttpClient client = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_3)
    .build();
```

The `HttpClient.Version` enum conceptually contains:

```
HttpClient.Version
├── HTTP_1_1
├── HTTP_2
└── HTTP_3
```

HTTP/2 vs HTTP/3

Feature	HTTP/2	HTTP/3
Transport	TCP	QUIC
Underlying network protocol	TCP	UDP
Encryption	TLS	TLS integrated with QUIC
Streams	Yes	Yes

Multiplexing	Yes	Yes
Head-of-line blocking	TCP-level HOL possible	Avoids TCP-level HOL
Java 26 HttpClient	Supported	Supported
Java API	java.net.http	java.net.http

Asynchronous HTTP/3 Example

Java's HttpClient also supports asynchronous operations through `sendAsync()`.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class HTTP3ClientAPI {

    public static void main(String[] args) {

        HttpClient client = HttpClient.newBuilder()
            .version(HttpClient.Version.HTTP_3)
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://example.com"))
            .GET()
            .build();

        client.sendAsync(
            request,
            HttpResponse.BodyHandlers.ofString()
        )
        .thenAccept(response -> {

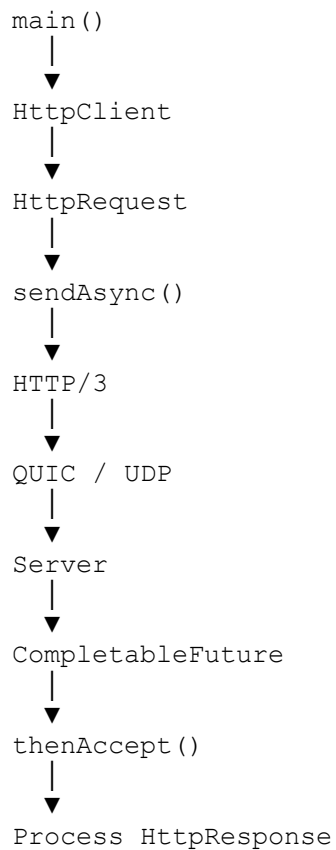
            System.out.println(
                "Status: " + response.statusCode()
            );

            System.out.println(
                "Version: " + response.version()
            );

            System.out.println(
                "Body: " + response.body()
            );

        })
        .join();
    }
}
```

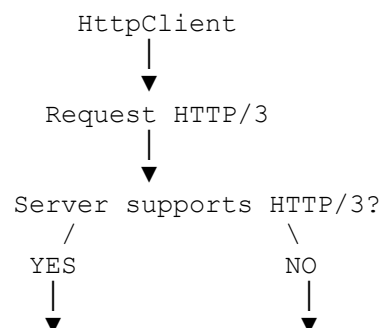
The asynchronous flow is:

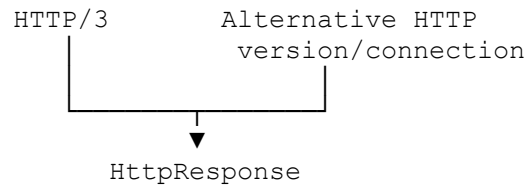


HTTP/3 Preference vs Fallback

One important aspect of the Java API is that HTTP version selection can involve negotiation/fallback behavior rather than blindly assuming that every destination supports HTTP/3.

Conceptually:





Therefore, simply requesting HTTP/3 does **not** mean every arbitrary HTTP server will necessarily communicate using HTTP/3.

Important Java APIs

The feature builds on the existing HTTP Client API:

```
java.net.http.HttpClient  
java.net.http.HttpRequest  
java.net.http.HttpResponse  
java.net.http.HttpClient.Version
```

The central addition is HTTP/3 support in:

```
HttpClient.Version.HTTP_3
```

This is particularly useful because existing Java HTTP-client application architecture can remain largely unchanged.

Why HTTP/3 Matters

1. Lower latency

QUIC was designed with fast connection establishment in mind.

2. Better multiplexing behavior

Multiple streams can operate independently within a QUIC connection.

3. Reduced head-of-line blocking

Loss affecting one QUIC stream does not impose the same TCP-level blocking behavior across unrelated streams.

4. Modern web compatibility

Many modern web services and CDNs increasingly support HTTP/3.

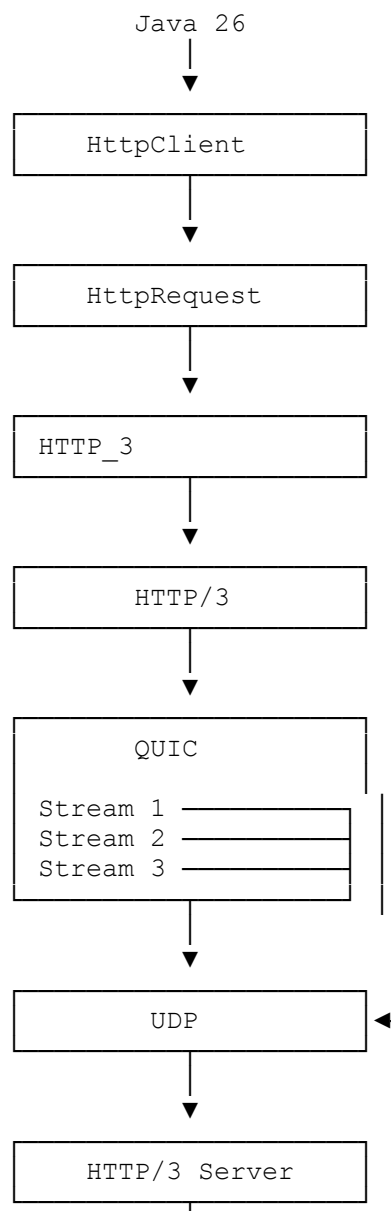
5. Existing Java API

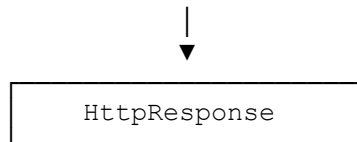
Developers can use the familiar:

HttpClient
HttpRequest
HttpResponse

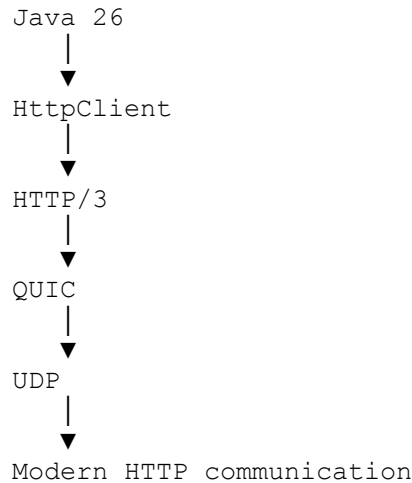
programming model rather than learning an entirely separate Java HTTP client API.

Complete Conceptual Architecture





Key Takeaway



One-line definition

JEP 517 in Java 26 adds HTTP/3 support to the standard `java.net.http.HttpClient` API, allowing Java applications to communicate using HTTP/3 over QUIC/UDP while retaining the familiar `HttpClient`, `HttpRequest`, and `HttpResponse` programming model.

Java 26 — G1 GC Throughput Improvements

Java Version: 26

Year: 2026

Feature: G1 GC Throughput Improvements

UniqueClassName: G1ThroughputSynchronization

JEP: 522 — G1 GC: Improve Throughput by Reducing Synchronization

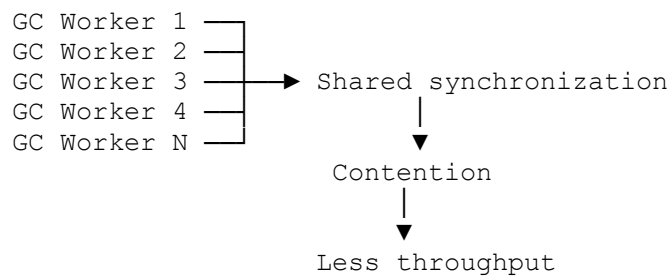
Definition

G1 GC Throughput Improvements improve the throughput of the **Garbage-First (G1) Garbage Collector** by reducing synchronization overhead during garbage-collection operations.

The main idea is to reduce contention caused when multiple GC worker threads access shared synchronization mechanisms. Instead of unnecessarily competing for shared locks/structures, G1 can use more efficient thread-local or less-contended mechanisms.

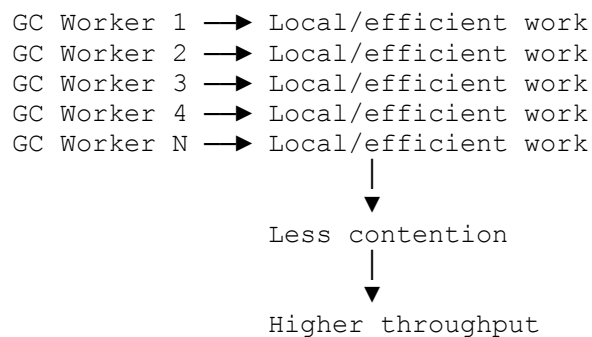
In simple terms:

Before Java 26



Java 26 improves this:

Java 26



What Is G1 GC?

G1 (Garbage-First) GC is a server-oriented garbage collector designed to provide a balance between:

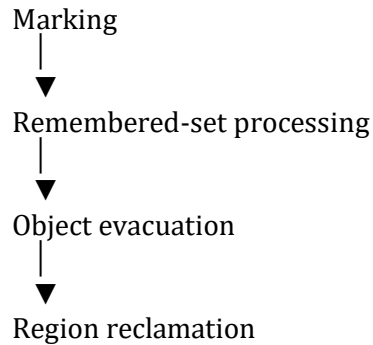
- application throughput,
- predictable pause times, and
- efficient heap management.

G1 divides the Java heap into many regions:

Java Heap

R1	R2	R3	R4	R5	R6	R7	R8
R9	R10	R11	R12	R13	R14	R15	R16

During collection, G1's GC worker threads perform tasks such as:

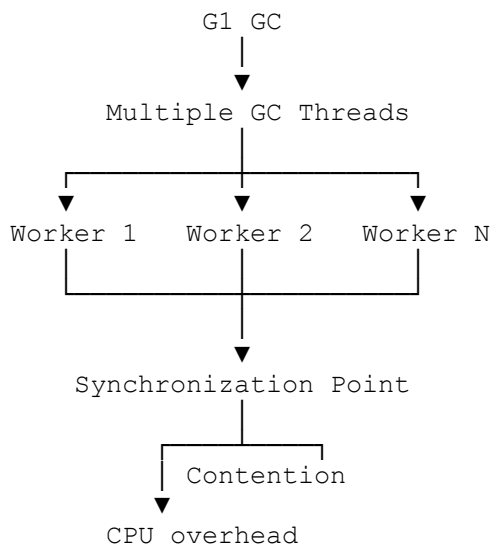


When many worker threads need to coordinate through shared synchronization points, synchronization can become a throughput bottleneck.

Java 26 Improvement

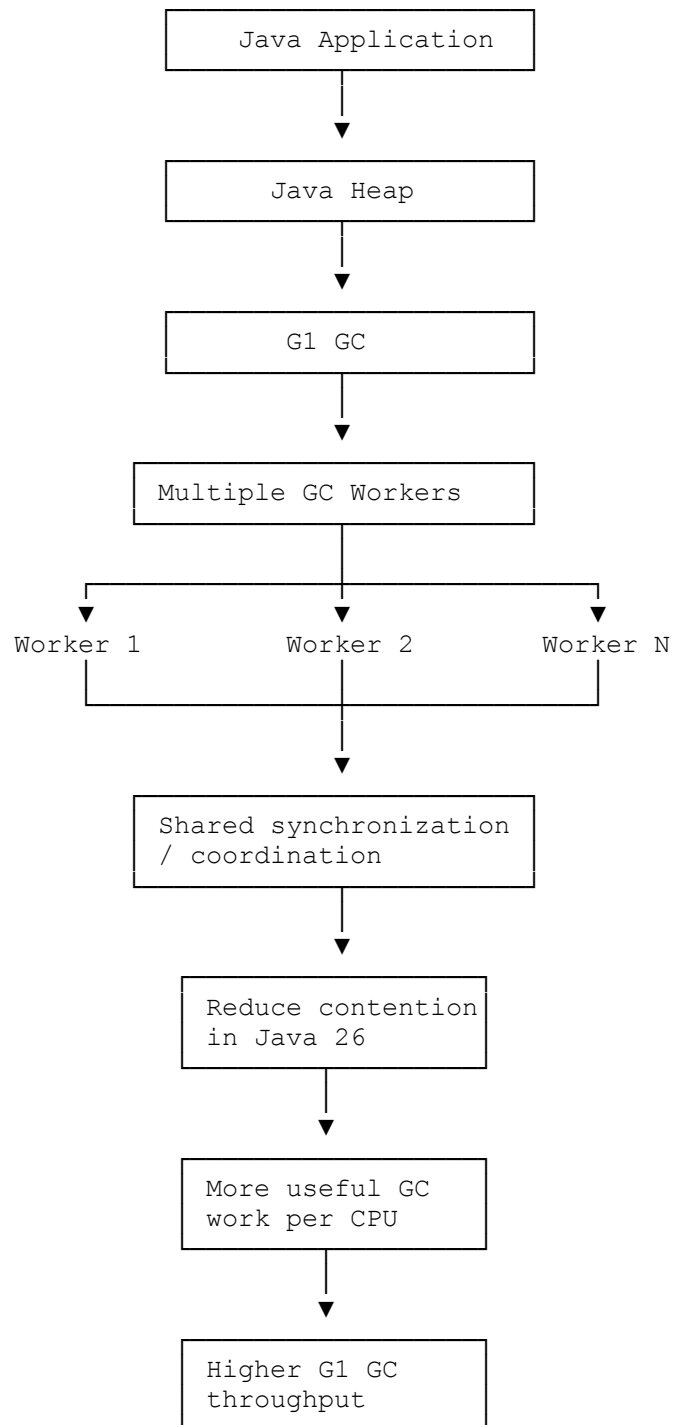
JEP 522 focuses on reducing synchronization overhead in G1's garbage-collection implementation.

Conceptually:



The optimization reduces unnecessary synchronization and allows GC worker threads to spend more time doing useful garbage-collection work.

Flowchart



Java Code Example

This is primarily a **JVM implementation optimization**, not a new Java language feature. Therefore, there is **no new Java keyword or API** that must be used.

A normal Java application automatically benefits when running with G1 on a Java 26 JVM.

```
import java.util.ArrayList;
import java.util.List;

public class G1ThroughputSynchronization {

    public static void main(String[] args) {

        System.out.println("G1 GC Throughput Demo");

        List<byte[]> objects = new ArrayList<>();

        for (int i = 0; i < 10_000; i++) {

            // Allocate temporary objects
            objects.add(new byte[1024]);

            // Periodically discard references
            // so that objects become eligible for GC.
            if (objects.size() > 1000) {
                objects.clear();
            }

        }

        System.out.println("Allocation completed.");
    }
}
```

The application code does not explicitly invoke the JEP 522 optimization.

The JVM's G1 implementation handles it internally.

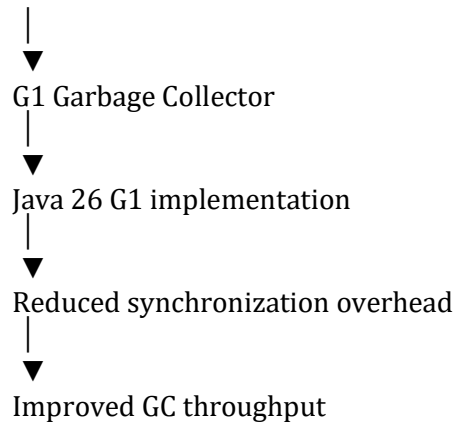
Running With G1

You can explicitly select G1:

```
java -XX:+UseG1GC G1ThroughputSynchronization
```

Conceptually:

```
java
|
└── -XX:+UseG1GC
```



Demonstrating GC Activity

For testing and benchmarking, you can enable GC logging:

```
java \
-XX:+UseG1GC \
-Xlog:gc* \
G1ThroughputSynchronization
```

This allows you to observe G1's collection behavior.

The important point is that **JEP 522 does not require special application-level code.**

Multithreaded Allocation Example

A workload that creates substantial allocation pressure can involve many application threads:

```
public class G1ThroughputSynchronization {

    private static final int THREAD_COUNT = 8;

    public static void main(String[] args)
        throws InterruptedException {

        Thread[] threads = new Thread[THREAD_COUNT];

        for (int i = 0; i < THREAD_COUNT; i++) {

            threads[i] = Thread.startVirtualThread(() -> {

                for (int j = 0; j < 100_000; j++) {

                    byte[] data = new byte[1024];
```

```

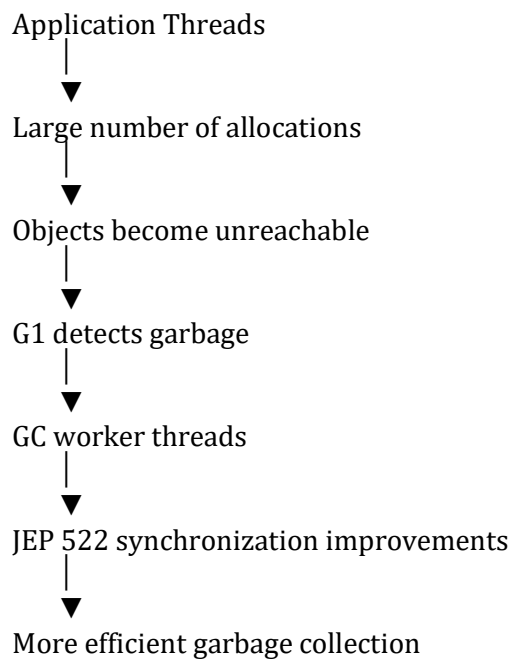
        // Use the object so the allocation
        // represents actual application work.
        data[0] = 1;
    }
    });
}

for (Thread thread : threads) {
    thread.join();
}

System.out.println("Work completed.");
}
}

```

Here:



Before vs Java 26

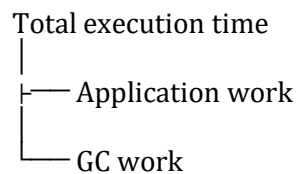
Aspect	Earlier G1	Java 26 G1
Garbage collector	G1	G1
Application code changes	None	None
GC worker synchronization	More overhead	Reduced overhead

Worker contention	Higher in affected paths	Reduced
CPU efficiency	Lower in affected workloads	Improved
GC throughput	Baseline	Improved
Main goal	Pause-time/throughput balance	Improve throughput further

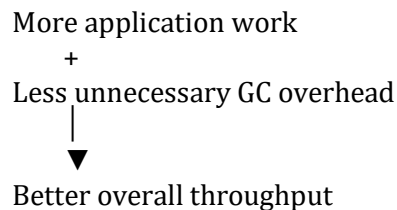
What Does "Throughput" Mean?

GC throughput is essentially about how much useful application work can be accomplished relative to the time/CPU resources spent on garbage collection.

Conceptually:

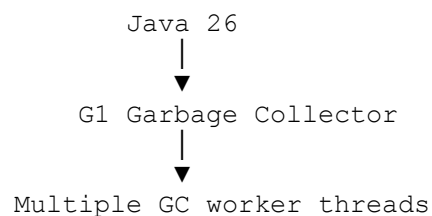


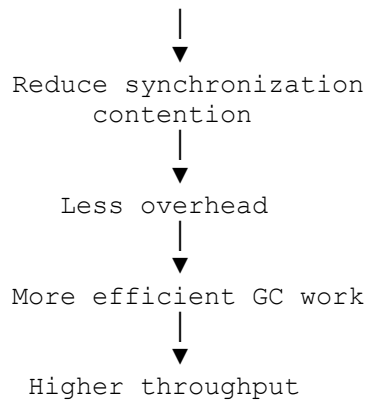
Better GC throughput means:



It does **not** mean that every application will automatically become faster by the same amount. The benefit depends on the workload, heap size, allocation rate, number of GC workers, hardware, and other JVM conditions.

Key Takeaway





One-line definition

JEP 522 in Java 26 improves G1 Garbage Collector throughput by reducing synchronization overhead among GC worker threads, allowing more CPU time to be spent on useful garbage-collection work rather than coordination and contention.

Java 26 — PEM Encodings of Cryptographic Objects (Second Preview)

Java Version: 26

Year: 2026

Feature: PEM Encodings of Cryptographic Objects (Second Preview)

UniqueClassName: PEMEncodings

JEP: 470 — PEM Encodings of Cryptographic Objects (Second Preview)

Definition

PEM Encodings of Cryptographic Objects provides a standard Java API for encoding and decoding cryptographic objects using **PEM (Privacy-Enhanced Mail)** format.

PEM is a widely used text-based representation for cryptographic data such as:

- Public keys
- Private keys
- Certificates
- Certificate signing requests
- Other cryptographic objects

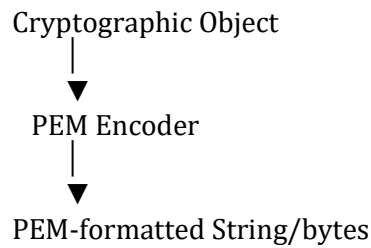
The feature makes it easier for Java applications to work directly with PEM data without requiring developers to manually implement Base64 encoding, header/footer processing, or format parsing.

Simple example

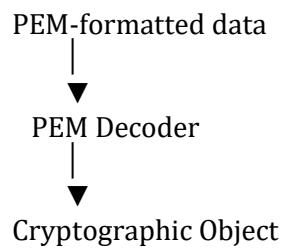
A PEM-encoded public key looks conceptually like:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
...
-----END PUBLIC KEY-----
```

Java 26 provides APIs intended to make this conversion easier:



and the reverse:



What Is PEM?

PEM is a textual encoding format commonly used by OpenSSL and other security tools.

Its basic structure is:

```
-----BEGIN <TYPE>-----
Base64 encoded DER data
-----END <TYPE>-----
```

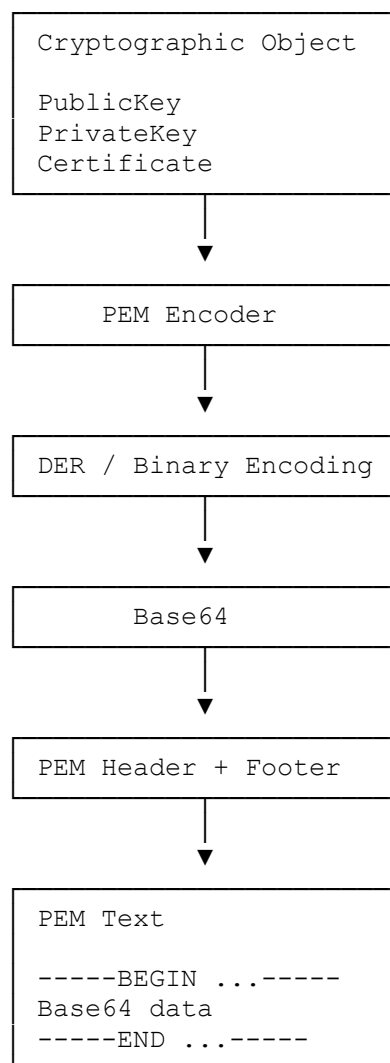
For example:

```
-----BEGIN PUBLIC KEY-----
<Base64 data>
-----END PUBLIC KEY-----
```

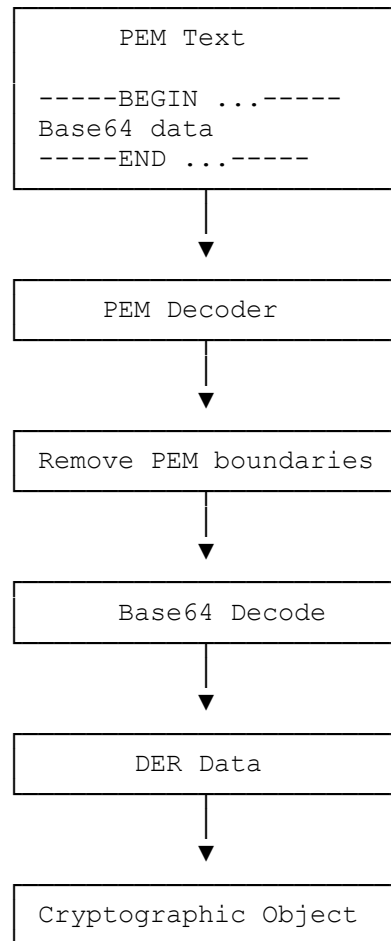
The underlying structure is approximately:

PEM Header -----BEGIN PUBLIC KEY-----
Base64 encoded DER MIIBIjANBgkqhki...
PEM Footer -----END PUBLIC KEY-----

Flowchart — Encoding



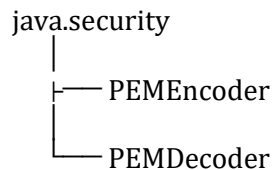
Flowchart — Decoding



Java 26 API Concept

The feature introduces the `java.security.PEMEncoder` and `java.security.PEMDecoder` APIs.

Conceptually:



The encoder converts supported cryptographic objects into PEM representations, while the decoder converts PEM data back into supported cryptographic representations.

Java Code Example — PEM Encoding

A simple example using an RSA public key:

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PublicKey;
import java.security.PEMEncoder;

public class PEMEncodings {

    public static void main(String[] args) throws Exception {

        // Generate an RSA key pair
        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair = generator.generateKeyPair();

        PublicKey publicKey = keyPair.getPublic();

        // Create PEM encoder
        PEMEncoder encoder = PEMEncoder.of();

        // Encode the public key as PEM
        String pem = encoder.encodeToString(publicKey);

        System.out.println(pem);
    }
}
```

The conceptual output is:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
...
-----END PUBLIC KEY-----
```

The actual Base64 content will be different each time because the generated key is different.

Java Code Example — PEM Decoding

The reverse operation can be performed with PEMDecoder.

```
import java.security.PublicKey;
import java.security.PEMDecoder;

public class PEMEncodings {
```

```

public static void main(String[] args) throws Exception {

    String pem = ""
        -----BEGIN PUBLIC KEY-----
        MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
        -----END PUBLIC KEY-----
        "";

    PEMDecoder decoder = PEMDecoder.of();

    Object object = decoder.decode(pem);

    System.out.println(
        "Decoded type: " + object.getClass().getName()
    );
}
}

```

For a real key, the Base64 content must be a complete, valid DER-encoded key. The shortened ... above is only illustrative.

Encoding and Decoding Together

A complete conceptual example:

```

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PEMDecoder;
import java.security.PEMEncoder;
import java.security.PublicKey;

public class PEMEncodings {

    public static void main(String[] args) throws Exception {

        // 1. Generate key pair
        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair = generator.generateKeyPair();

        PublicKey originalKey = keyPair.getPublic();

        // 2. Encode PublicKey -> PEM
        PEMEncoder encoder = PEMEncoder.of();

        String pem =
            encoder.encodeToString(originalKey);
    }
}

```

```

        System.out.println("PEM:");
        System.out.println(pem);

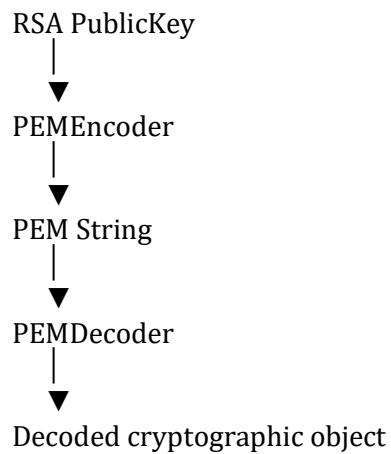
        // 3. Decode PEM -> Object
        PEMDecoder decoder = PEMDecoder.of();

        Object decoded =
            decoder.decode(pem);

        // 4. Display result
        System.out.println(
            "Decoded object: "
            + decoded.getClass().getName()
        );
    }
}

```

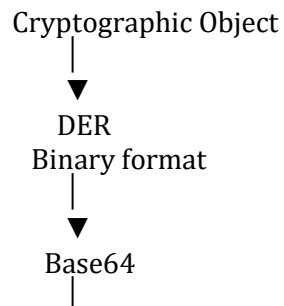
The data flow is:



PEM and DER Relationship

PEM should not be confused with the underlying cryptographic encoding.

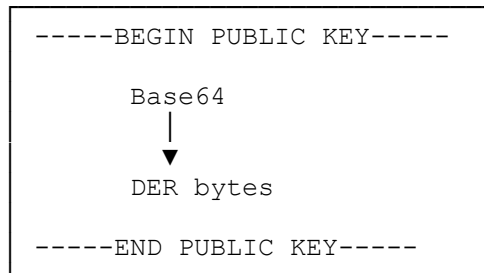
Typically:



▼
PEM envelope

Therefore, PEM is essentially a convenient **textual representation around encoded binary cryptographic data**.

For example:



Common PEM Types

PEM Type	Typical Object
PUBLIC KEY	Public key
PRIVATE KEY	PKCS#8 private key
CERTIFICATE	X.509 certificate
CERTIFICATE REQUEST	PKCS#10 CSR
ENCRYPTED PRIVATE KEY	Encrypted private key

The Java API provides a standardized way to work with supported PEM representations rather than requiring application code to manually manipulate these delimiters and Base64 content.

Why Is This Useful?

Before

Developers often had to manually perform operations such as:

```
String pem =  
    "-----BEGIN PUBLIC KEY-----\n"  
    + Base64.getEncoder().encodeToString(encoded)
```

```
+ "\n-----END PUBLIC KEY-----";
```

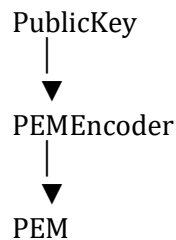
This approach requires application code to understand PEM formatting.

With the PEM API

The operation becomes conceptually:

```
String pem =  
    PEMEncoder.of().encodeToString(publicKey);
```

This makes the intent much clearer:



Practical Use Cases

The API is particularly useful when Java applications interact with:

- TLS certificates and keys
- OpenSSL-generated files
- PKI systems
- Certificate authorities
- JWT/JWS signing infrastructure
- SSH/security infrastructure where supported formats overlap
- Cloud key-management workflows
- Configuration files containing certificates or keys
- REST APIs that exchange PEM-formatted cryptographic material

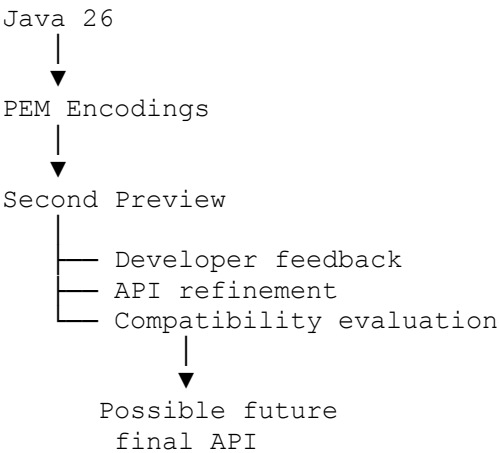
Second Preview

The phrase "**Second Preview**" is important.

It means this API is being exposed as a **preview feature** in Java 26 for further evaluation and refinement rather than being treated as an ordinary permanent API.

When compiling/running code using preview functionality, the appropriate preview compiler/runtime options may be required depending on the exact JDK 26 build and API usage.

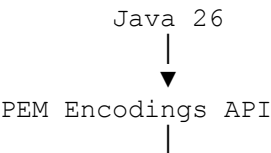
Conceptually:

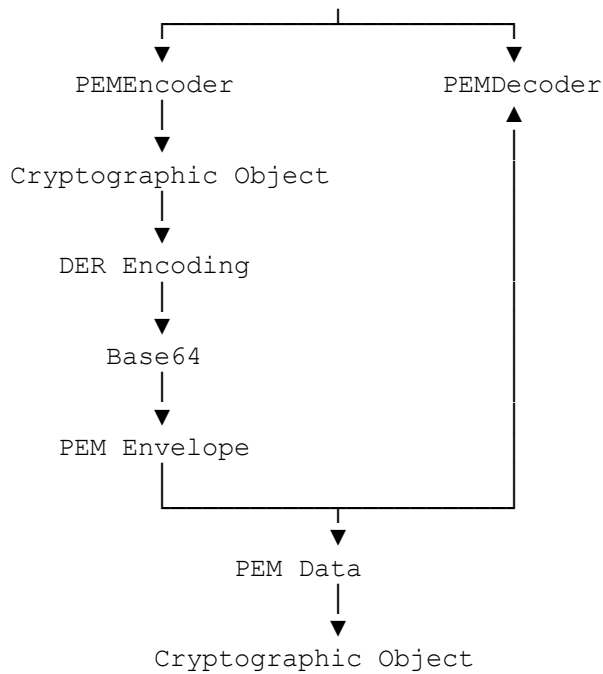


Before vs Java 26

Aspect	Traditional Java approach	Java 26 PEM API
PEM creation	Manual formatting/Base64 or third-party library	Standard JDK API
PEM parsing	Manual parsing/library	Standard JDK API
Base64 handling	Developer responsibility	Encapsulated by API
PEM headers/footers	Developer responsibility	Encapsulated
Cryptographic integration	Separate code	Integrated with security APIs
API status	No standard PEM API	Second Preview
Main classes	—	PEMEncoder, PEMDecoder

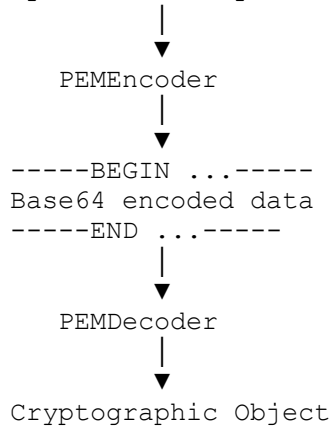
Complete Flow





Key Takeaway

`PublicKey` / `PrivateKey` / `Certificate`



One-line definition

Java 26's "PEM Encodings of Cryptographic Objects (Second Preview)" introduces standard `PEMEncoder` and `PEMDecoder` APIs for converting supported cryptographic objects to and from widely used PEM representations, eliminating much of the need for application-specific PEM formatting and parsing code.

Java 26 — Structured Concurrency (Sixth Preview)

Java Version: 26

Year: 2026

Feature: Structured Concurrency (Sixth Preview)

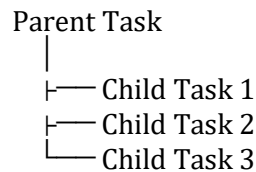
UniqueClassName: StructuredConcurrency

JEP: 525 — Structured Concurrency (Sixth Preview)

Definition

Structured Concurrency treats a group of related concurrent tasks as a single unit of work.

Instead of creating independent threads/tasks and manually coordinating them, structured concurrency creates a clear **parent-child relationship** between concurrent tasks:



The parent task remains responsible for its children.

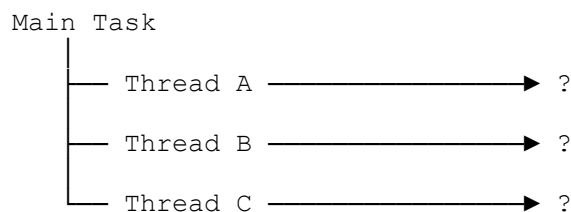
The key idea is:

Concurrent work should have a well-defined lifetime and ownership relationship, just like structured blocks in sequential code.

Java 26 continues the evolution of Structured Concurrency as the **sixth preview**.

Why Structured Concurrency?

Traditional concurrent code can become difficult to manage:



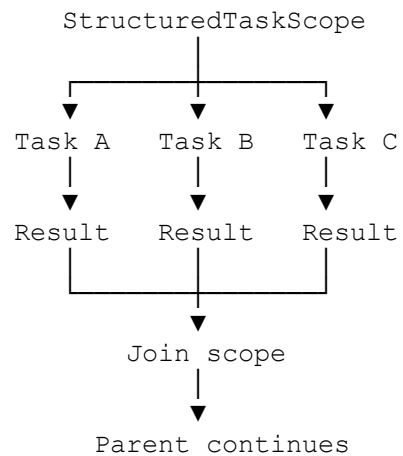
Questions arise:

- Did all tasks finish?
- What happens if one fails?

- Should the remaining tasks be cancelled?
- Which exception should be propagated?
- Are any threads still running?
- Who owns the child tasks?

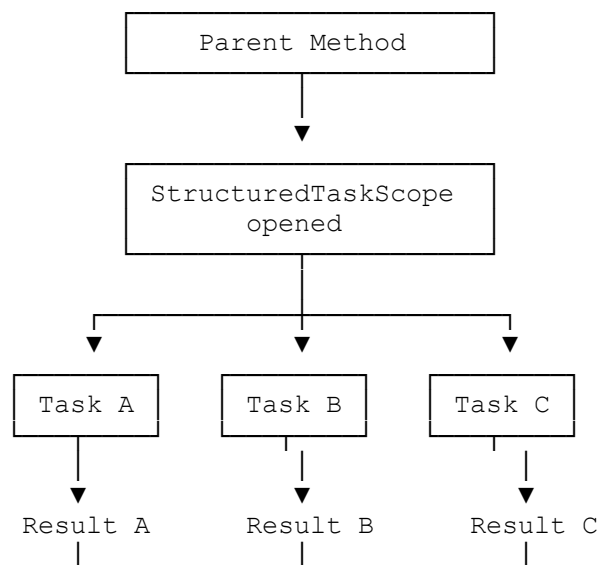
Structured concurrency provides a scope that answers these questions.

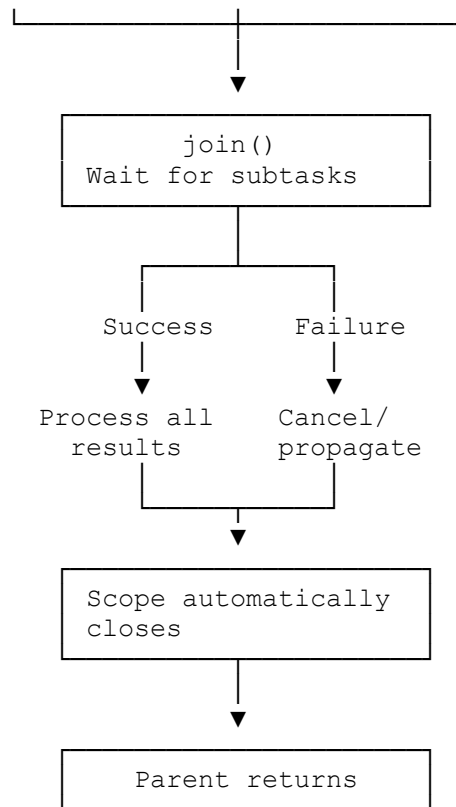
Basic Model



The lifetime of the child tasks is bounded by the lifetime of the structured-concurrency scope.

Flowchart





Java 26 Code Example

Because Structured Concurrency is a **preview feature**, compile and run the example with preview enabled.

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws Exception {

        try (var scope =
            StructuredTaskScope.open()) {

            var userTask = scope.fork(
                () -> fetchUser()
            );

            var ordersTask = scope.fork(
                () -> fetchOrders()
            );

            scope.join();
```

```

        String user = userTask.get();
        String orders = ordersTask.get();

        System.out.println("User: " + user);
        System.out.println("Orders: " + orders);
    }
}

static String fetchUser() throws InterruptedException {
    Thread.sleep(500);
    return "User-101";
}

static String fetchOrders() throws InterruptedException {
    Thread.sleep(700);
    return "Orders: [Order-1, Order-2]";
}
}

```

The important part is:

```

try (var scope = StructuredTaskScope.open()) {

    var userTask = scope.fork(
        () -> fetchUser()
    );

    var ordersTask = scope.fork(
        () -> fetchOrders()
    );

    scope.join();
}

```

How the Example Works

Step 1 — Open the scope

```
var scope = StructuredTaskScope.open();
```

This creates the structured concurrency context.

Parent



StructuredTaskScope

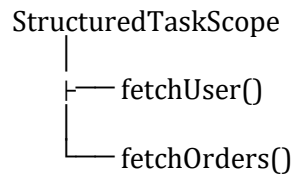
Step 2 — Fork child tasks

```
var userTask = scope.fork(  
    () -> fetchUser()  
);
```

and:

```
var ordersTask = scope.fork(  
    () -> fetchOrders()  
);
```

Now we have:

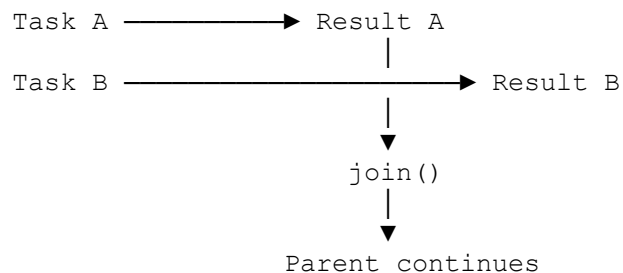


The tasks can execute concurrently.

Step 3 — Join

```
scope.join();
```

The parent waits for the child tasks.



Step 4 — Retrieve results

```
String user = userTask.get();  
String orders = ordersTask.get();
```

The parent obtains the results of the child computations.

Parallel Execution

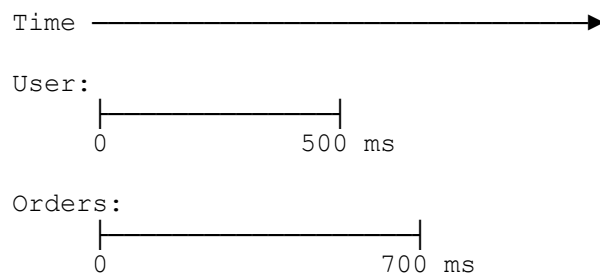
Suppose:

fetchUser() = 500 ms
fetchOrders() = 700 ms

Sequential execution would approximately take:

500 ms + 700 ms
↓
1200 ms

Structured concurrency allows the independent operations to execute concurrently:



Total is approximately the longest task:

≈ 700 ms

rather than:

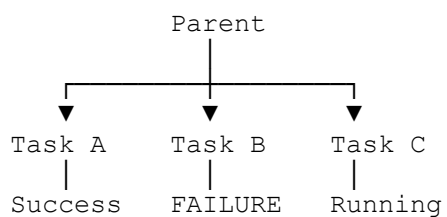
≈ 1200 ms

Actual performance depends on the workload and scheduling.

Failure Handling

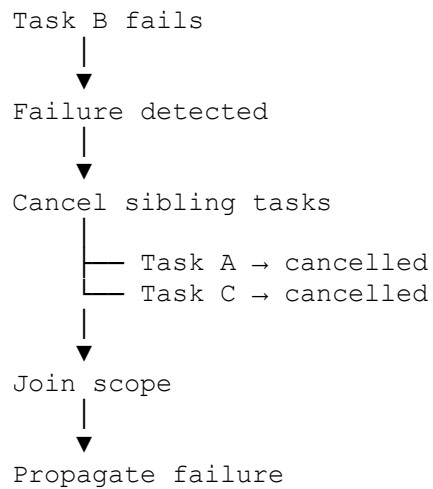
One of the major advantages is structured failure management.

Imagine:



A structured concurrency policy can respond to the failure by cancelling sibling tasks.

Conceptually:



This prevents child tasks from accidentally continuing after the parent operation has already failed.

Example — Shutdown on Failure

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws Exception {

        try (var scope =
            StructuredTaskScope.open()) {

            var task1 = scope.fork(
                () -> successfulTask()
            );

            var task2 = scope.fork(
                () -> failingTask()
            );

            scope.join();

            System.out.println(task1.get());
            System.out.println(task2.get());

        }
    }
}
```

```

static String successfulTask()
    throws InterruptedException {

    Thread.sleep(1000);

    return "Task 1 completed";
}

static String failingTask() {

    throw new RuntimeException(
        "Task 2 failed"
    );
}
}

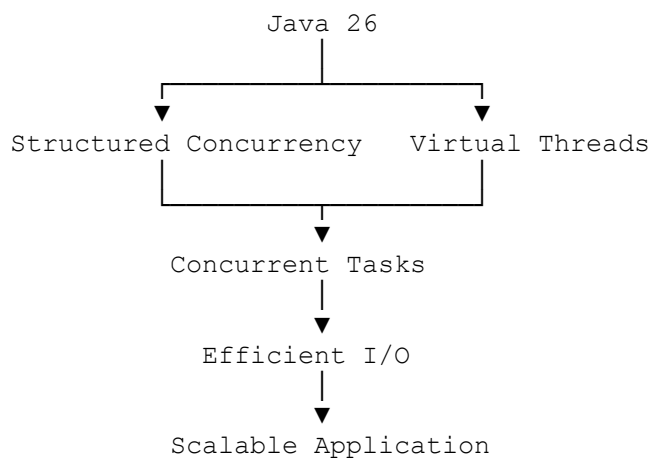
```

The scope gives the application a structured place to coordinate these operations.

Structured Concurrency + Virtual Threads

Structured concurrency works especially well with **virtual threads**.

The conceptual architecture is:



For example:

```

import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws Exception {

```

```

try (var scope =
    StructuredTaskScope.open()) {

    var task1 = scope.fork(
        () -> {
            System.out.println(
                "Task 1: "
                + Thread.currentThread()
            );
            return "A";
        }
    );

    var task2 = scope.fork(
        () -> {
            System.out.println(
                "Task 2: "
                + Thread.currentThread()
            );
            return "B";
        }
    );

    scope.join();

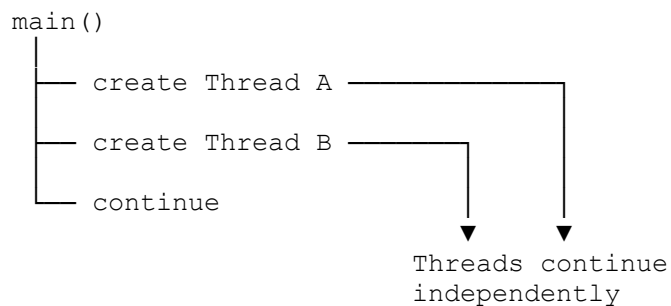
    System.out.println(
        task1.get() + task2.get()
    );
}
}

```

The actual execution mechanism is managed by the structured-concurrency API and the JVM's thread implementation.

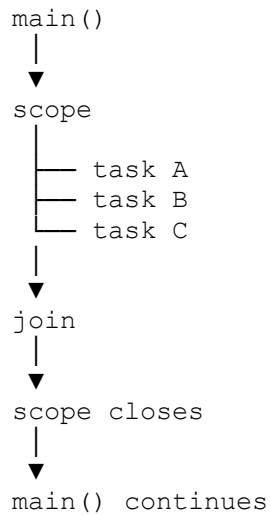
Structured vs Unstructured Concurrency

Traditional approach



The relationship between parent and child work can become unclear.

Structured approach



The lifetime and ownership relationship is explicit.

Key Principles

Principle	Meaning
Task hierarchy	Child tasks belong to a structured scope
Bounded lifetime	Child tasks should not outlive their scope
Join	Parent explicitly waits for child completion
Failure propagation	Failure can be managed as part of the scope
Cancellation	Related tasks can be cancelled together
Observability	The task hierarchy makes concurrent operations easier to understand
Composition	Multiple concurrent operations can be treated as one unit

Sixth Preview

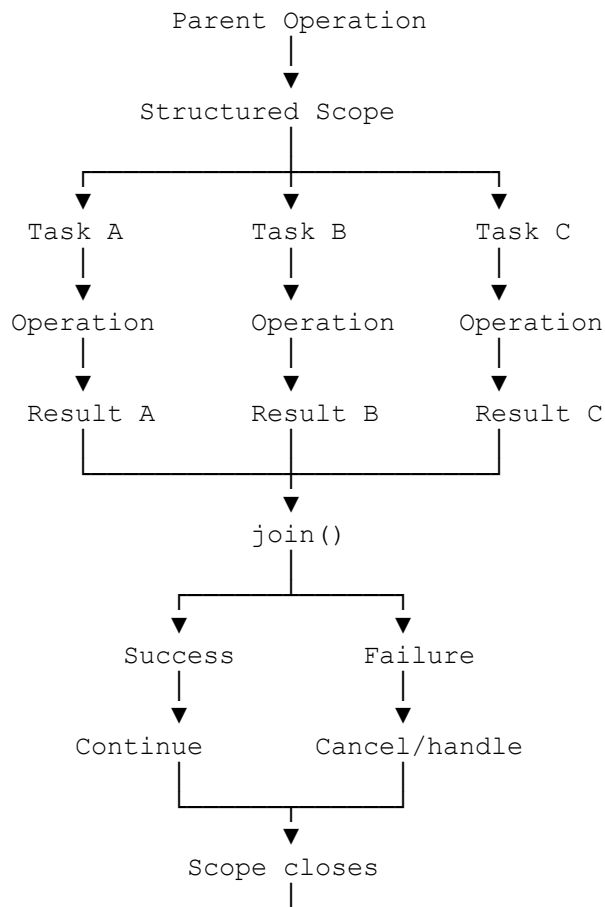
The feature has evolved through multiple previews:

Java 19 → First Preview
Java 20 → Second Preview
Java 21 → Third Preview
Java 22 → Fourth Preview
Java 23 → Fifth Preview
Java 24 → Sixth Preview
...

However, **the version mapping matters here**: the feature's preview count should be checked against the specific JEP/version history rather than inferred simply from the number of Java releases. For Java 26, **JEP 525 is the sixth preview**.

The API continues to evolve during preview stages, so code written against an earlier preview may require changes when moving to Java 26.

Complete Flow



▼
Parent continues

One-line Definition

Java 26 Structured Concurrency (JEP 525) provides a structured model for running and coordinating related concurrent tasks as a single unit, giving those tasks clear ownership, bounded lifetimes, coordinated joining, cancellation, and failure handling.

Java 26 — Lazy Constants (Second Preview)

Java Version: 26

Year: 2026

Feature: Lazy Constants (Second Preview)

UniqueClassName: LazyConstants

JEP: 526 — Lazy Constants (Second Preview)

Definition

Lazy Constants introduce a mechanism for declaring values that are **computed lazily**—only when they are first needed—while still allowing the JVM to treat them as stable constants after initialization.

The key idea is to combine:

Lazy initialization

+

Constant-like semantics

=

Lazy Constant

A lazy constant differs from a normal static final field because its value does not have to be created during class initialization.

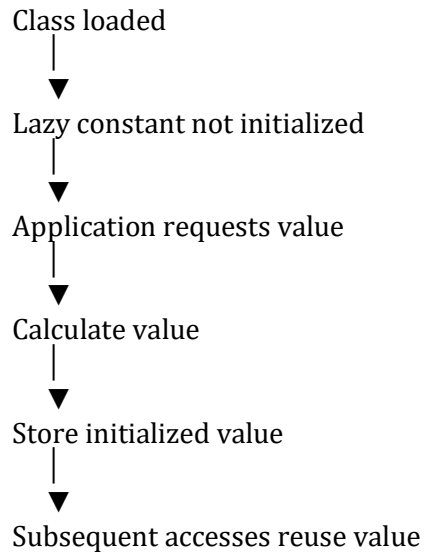
Traditional static final

```
static final String VALUE = expensiveCalculation();
```

The calculation happens as part of class initialization.

Lazy constant

Conceptually:



This can be particularly useful for expensive objects that may never actually be used.

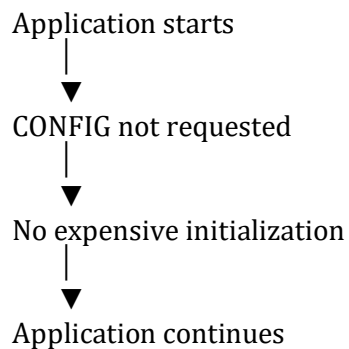
Why Lazy Constants?

Consider:

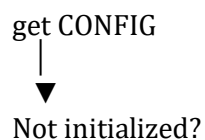
```
public static final Config CONFIG =  
    loadLargeConfiguration();
```

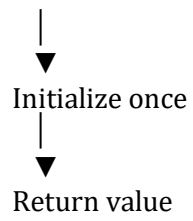
If CONFIG is never used, the application may still perform the expensive initialization.

With lazy initialization:

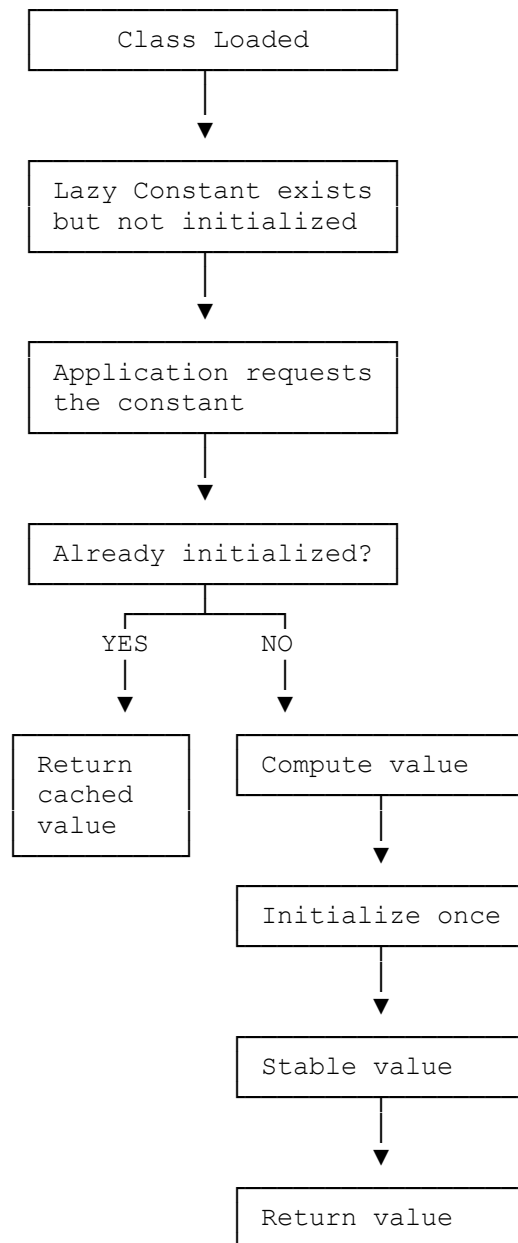


When it is finally needed:





Flowchart

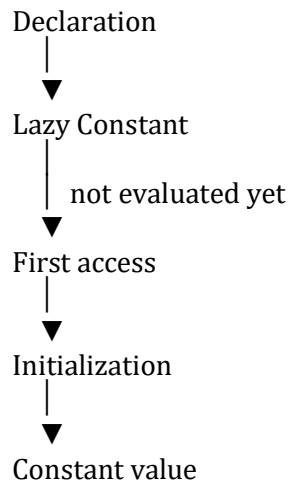


Important Concept

A normal static final field combines initialization and immutability:

```
static final String NAME = createName();
```

But a lazy constant separates **declaration** from **initialization**:



The objective is to let the JVM understand that, once initialized, the value is stable and can potentially be optimized accordingly.

Java Code Example

JEP 526 introduces the `java.lang.LazyConstant` API.

A basic example is conceptually:

```
import java.lang.LazyConstant;

public class LazyConstants {

    private static final LazyConstant<String> MESSAGE =
        LazyConstant.of(
            () -> createMessage()
        );

    private static String createMessage() {

        System.out.println(
            "Creating message..."
        );

        return "Hello from Java 26!";
    }
}
```

```

public static void main(String[] args) {

    System.out.println("Application started.");

    System.out.println(
        "Message has not been requested yet."
    );

    // First access
    System.out.println(MESSAGE.get());

    // Second access
    System.out.println(MESSAGE.get());

}
}

```

Conceptually, the output is:

```

Application started.
Message has not been requested yet.
Creating message...
Hello from Java 26!
Hello from Java 26!

```

Notice:

Creating message...

appears only once.

How It Works

The declaration:

```

private static final LazyConstant<String> MESSAGE =
    LazyConstant.of(
        () -> createMessage()
    );

```

does **not** immediately invoke:

```
createMessage()
```

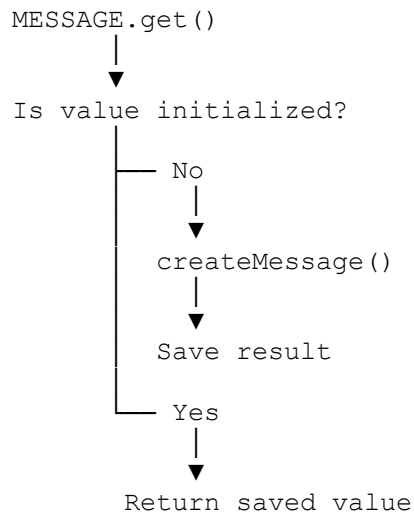
Instead, it provides the computation:

```
() -> createMessage()
```

The computation is evaluated when the value is requested.

`MESSAGE.get()`

causes:



Example — Expensive Object

Lazy constants can be useful when constructing an object is expensive.

```
import java.lang.LazyConstant;
import java.util.Map;

public class LazyConstants {

    private static final LazyConstant<Map<String, String>>
        CONFIG =
            LazyConstant.of(
                LazyConstants::loadConfiguration
            );

    private static Map<String, String>
    loadConfiguration() {

        System.out.println(
            "Loading configuration..."
        );

        return Map.of(
            "environment", "production",
            "region", "ap-south-1",
            "version", "26"
        );
    }
}
```

```

public static void main(String[] args) {

    System.out.println("Application started.");

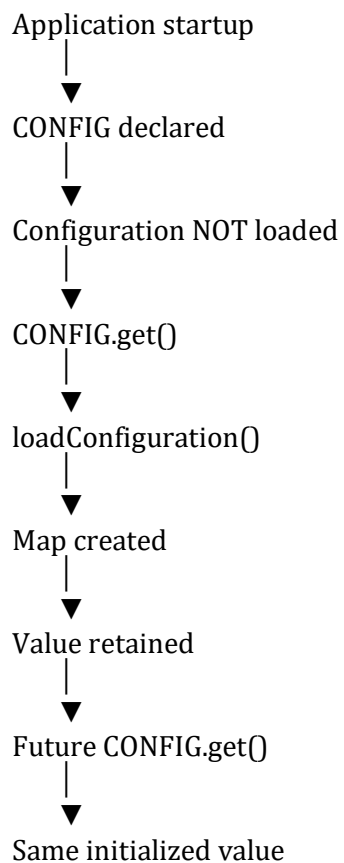
    // Configuration is loaded only here.
    Map<String, String> config =
        CONFIG.get();

    System.out.println(
        "Environment = "
        + config.get("environment")
    );

    // Reuses the initialized value.
    System.out.println(
        "Region = "
        + CONFIG.get().get("region")
    );
}
}

```

Conceptual flow:



Lazy Constant vs static final

Feature	static final	Lazy Constant
Immutable after initialization	Yes	Yes
Initialization timing	Usually class initialization	First access
Expensive computation can be delayed	No	Yes
Can avoid unused initialization	No	Yes
JVM-aware constant semantics	Traditional field semantics	Designed for stable lazy values
Thread-safe initialization	Class initialization provides guarantees	API manages initialization semantics
Java 26	Existing	Preview feature

Lazy Constant vs Supplier

You might ask:

Why not simply use `Supplier<T>`?

For example:

```
static final Supplier<String> VALUE =  
    () -> expensiveCalculation();
```

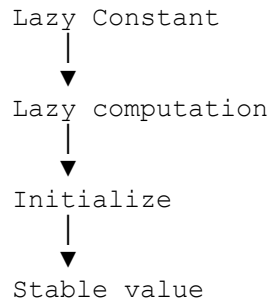
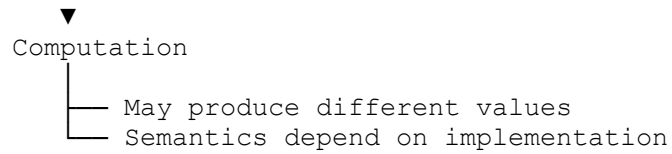
A `Supplier` represents a computation, but it does not inherently communicate the same **constant-like semantic** to the JVM.

A lazy constant is intended to tell the JVM:

"This value is computed lazily,
but once initialized it is stable."

Conceptually:

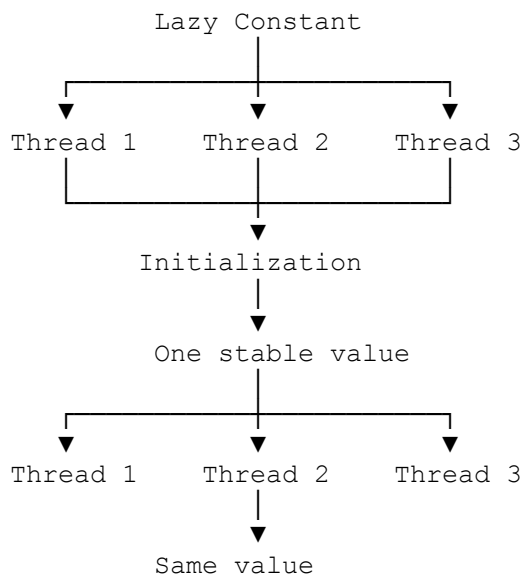
```
Supplier  
|
```



This distinction is important for JVM optimization.

Thread-Safety Concept

Suppose multiple threads access the constant simultaneously:



The API is designed to provide appropriate initialization and publication semantics rather than requiring developers to build their own synchronization mechanism.

Another Example — Database Configuration

```
import java.lang.LazyConstant;

public class LazyConstants {

    private static final LazyConstant<String>
        DATABASE_URL =
        LazyConstant.of(
            () -> loadDatabaseUrl()
        );

    private static String loadDatabaseUrl() {

        System.out.println(
            "Reading database configuration..."
        );

        return "jdbc:postgresql://localhost:5432/app";
    }

    public static void main(String[] args) {

        System.out.println(
            "Starting application..."
        );

        // Other application startup work...

        String url = DATABASE_URL.get();

        System.out.println(
            "Database URL: " + url
        );
    }
}
```

The database configuration is not loaded until:

`DATABASE_URL.get()`

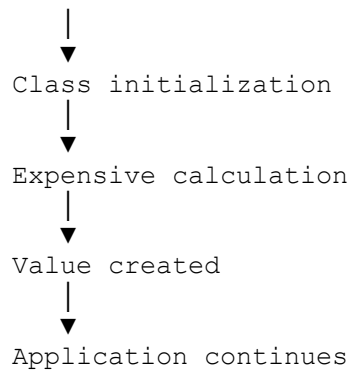
is executed.

Performance Motivation

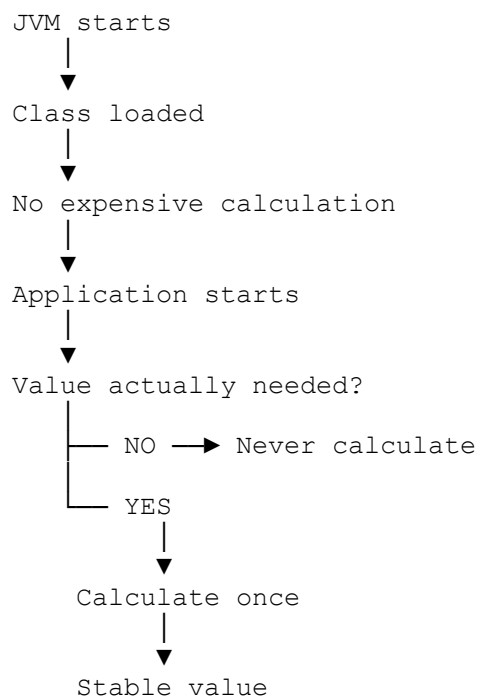
The motivation can be represented as:

Traditional static initialization

JVM starts



With lazy constants:



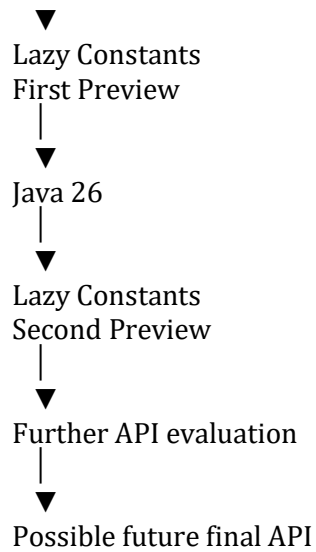
This can improve startup time and reduce unnecessary allocation/work when a value is rarely used.

Sixth Preview

The feature is a **preview API**, meaning it is still subject to refinement.

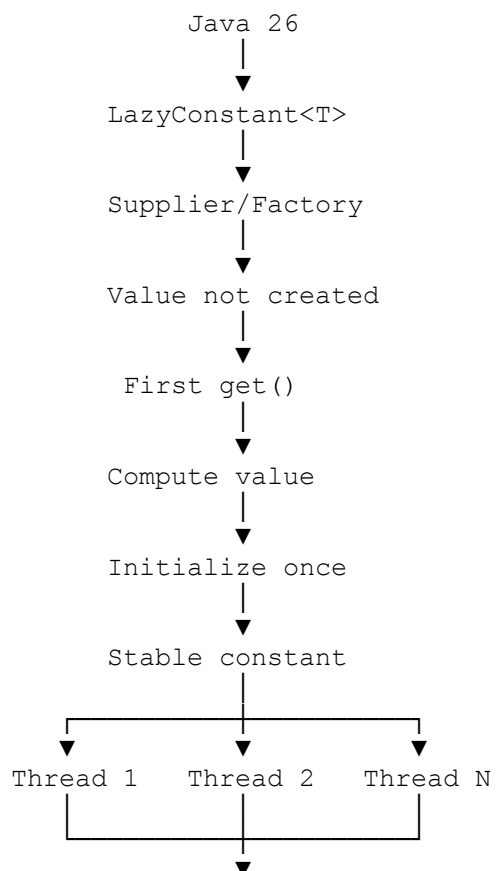
Its evolution is:

Java 25
|



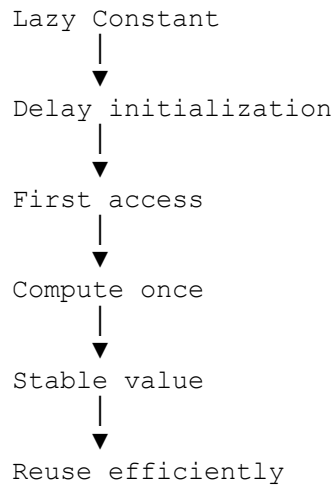
Therefore, code using the Java 26 preview API should be compiled and run with preview features enabled, according to the JDK 26 toolchain.

Complete Architecture



Same stable value

Key Takeaway



One-line definition

Java 26's Lazy Constants (JEP 526, Second Preview) provide a JVM-aware mechanism for lazily computing a value on first access and then treating the resulting value as stable, combining the benefits of lazy initialization with constant-like semantics.

Java 26 — Vector API (Eleventh Incubator)

Java Version: 26

Year: 2026

Feature: Vector API (Eleventh Incubator)

UniqueClassName: VectorAPI

JEP: 529 — Vector API (Eleventh Incubator)

Definition

The **Vector API** provides a Java API for expressing **vector computations** that can be compiled by the JIT compiler into efficient SIMD (Single Instruction, Multiple Data) instructions supported by the underlying CPU.

Instead of processing values one at a time:

Scalar

```
a1 → operation → r1
a2 → operation → r2
a3 → operation → r3
a4 → operation → r4
```

Vector processing can operate on multiple values simultaneously:

SIMD / Vector

```
[a1 a2 a3 a4]
  ↓
[operation]
  ↓
[r1 r2 r3 r4]
```

The API is designed to let Java developers write vectorized algorithms while allowing the JVM to select appropriate hardware instructions for the target CPU.

Why Vector API?

Consider adding two arrays.

Traditional Java

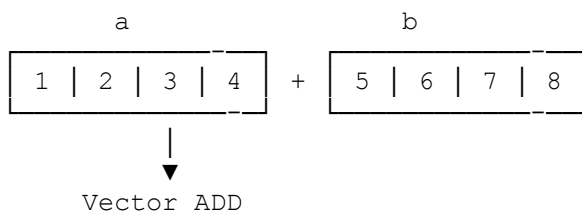
```
for (int i = 0; i < length; i++) {
    result[i] = a[i] + b[i];
}
```

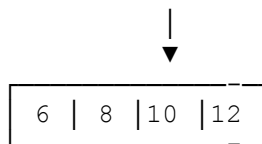
Conceptually:

```
a[0] + b[0] → result[0]
a[1] + b[1] → result[1]
a[2] + b[2] → result[2]
a[3] + b[3] → result[3]
```

Vector API

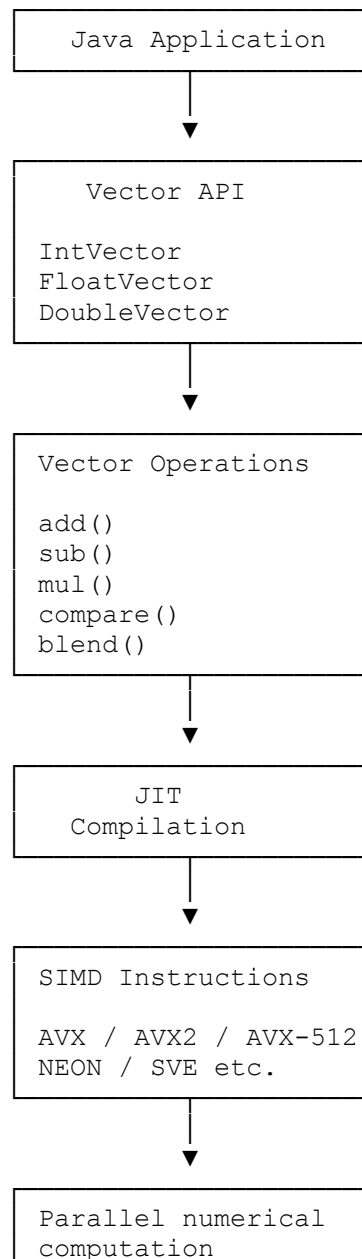
Several elements can be processed in one vector operation:





The exact number of elements processed together depends on the vector species and hardware.

Flowchart



Important API Classes

The Vector API is primarily located in:

`jdk.incubator.vector`

Important types include:

Type	Purpose
VectorSpecies<T>	Describes a vector shape/type supported by the platform
IntVector	Vector of int values
LongVector	Vector of long values
FloatVector	Vector of float values
DoubleVector	Vector of double values
VectorMask<T>	Represents which vector lanes are active
VectorOperators	Provides vector operations

Java Code Example — Integer Addition

```
import jdk.incubator.vector.IntVector;
import jdk.incubator.vector.VectorSpecies;

public class VectorAPI {

    private static final VectorSpecies<Integer> SPECIES =
        IntVector.SPECIES_PREFERRED;

    public static void main(String[] args) {

        int[] a = {
            1, 2, 3, 4,
            5, 6, 7, 8
        };

        int[] b = {
            10, 20, 30, 40,
```

```

        50, 60, 70, 80
    };

    int[] result = new int[a.length];

    int i = 0;

    // Process complete vector chunks
    for (; i < SPECIES.loopBound(a.length);
        i += SPECIES.length()) {

        IntVector va =
            IntVector.fromArray(SPECIES, a, i);

        IntVector vb =
            IntVector.fromArray(SPECIES, b, i);

        IntVector vr =
            va.add(vb);

        vr.intoArray(result, i);
    }

    // Process remaining elements
    for (; i < a.length; i++) {
        result[i] = a[i] + b[i];
    }

    for (int value : result) {
        System.out.print(value + " ");
    }
}

```

Output

11 22 33 44 55 66 77 88

How the Code Works

1. Select the preferred vector species

`IntVector.SPECIES_PREFERRED`

This allows the JVM to select an appropriate vector size for the current platform.

CPU
 |
 |—— Vector width available
 |

▼
SPECIES_PREFERRED
|
▼
Vector length selected

This is important because Java code doesn't need to hard-code a particular SIMD width.

2. Load array data

```
IntVector va =  
    IntVector.fromArray(SPECIES, a, i);
```

This loads multiple int values into one vector.

For example:

```
Array  
-----  
[1] [2] [3] [4] [5] [6] [7] [8]  
  |  
  ▼  
Vector  
-----  
[1  2  3  4]
```

The actual width depends on the selected species.

3. Perform vector operation

```
IntVector vr = va.add(vb);
```

Instead of:

```
1 + 10  
2 + 20  
3 + 30  
4 + 40
```

individually, the conceptual vector operation is:

```
[1  2  3  4]  
  +  
[10 20 30 40]  
  |
```

▼
[11 22 33 44]

4. Store the result

```
vr.toArray(result, i);
```

The vector is written back to the Java array.

```
Vector  
[11 22 33 44]  
  |  
  ▼  
Array  
[11][22][33][44]
```

Vectorized Multiplication Example

The same approach works for multiplication:

```
import jdk.incubator.vector.IntVector;  
import jdk.incubator.vector.VectorSpecies;  
  
public class VectorAPI {  
  
    private static final VectorSpecies<Integer> SPECIES =  
        IntVector.SPECIES_PREFERRED;  
  
    public static void main(String[] args) {  
  
        int[] values = {  
            1, 2, 3, 4,  
            5, 6, 7, 8  
        };  
  
        int[] result = new int[values.length];  
  
        IntVector multiplier =  
            IntVector.broadcast(SPECIES, 10);  
  
        int i = 0;  
  
        for (; i < SPECIES.loopBound(values.length);  
            i += SPECIES.length()) {  
  
            IntVector vector =  
                IntVector.fromArray(  
                    SPECIES,  
                    values,  
                    i
```

```

        );

        IntVector resultVector =
            vector.mul(multiplier);

        resultVector.intoArray(
            result,
            i
        );
    }

    for (; i < values.length; i++) {
        result[i] = values[i] * 10;
    }

    for (int value : result) {
        System.out.print(value + " ");
    }
}

```

Output:

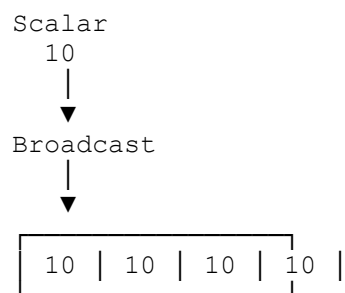
10 20 30 40 50 60 70 80

Vector Broadcast

This line:

```
IntVector.broadcast(SPECIES, 10);
```

creates a vector containing the same value in every lane.



Then:

```
vector.mul(multiplier);
```

conceptually becomes:

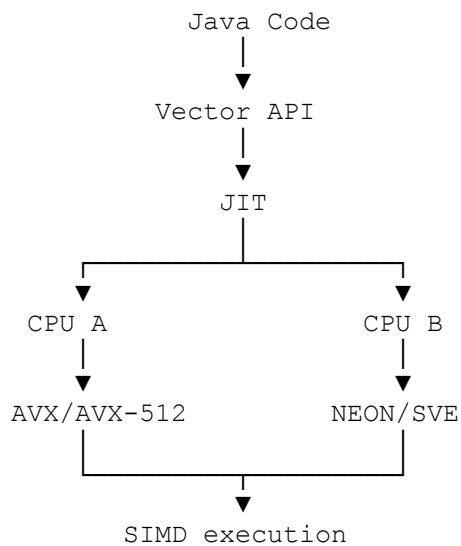
```

[1 2 3 4]
  ×
[10 10 10 10]
  ↓
[10 20 30 40]

```

Vector API + CPU

One of the major advantages is that application code does not have to explicitly write CPU-specific SIMD assembly.



The JVM can map vector operations to suitable hardware instructions when possible.

Scalar vs Vector

Scalar

for each element:

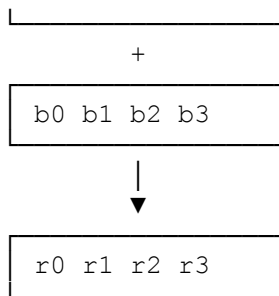
```

a[0] + b[0]
a[1] + b[1]
a[2] + b[2]
a[3] + b[3]

```

Vector

a0	a1	a2	a3
----	----	----	----



The vector approach can potentially process several elements per instruction.

Handling Remaining Elements

Arrays aren't necessarily an exact multiple of the vector size.

For example:

Array length = 10

Vector length = 4

Processing becomes:

Vector 1 → elements 0-3

Vector 2 → elements 4-7

Scalar → elements 8-9

That's why the example uses:

`SPECIES.loopBound(a.length)`

followed by a normal scalar loop:

```

for (; i < a.length; i++) {
    result[i] = a[i] + b[i];
}
  
```

Conceptually:

Array

0	1	2	3	→ Vector
4	5	6	7	→ Vector
8	9			→ Scalar

Vector Masks

Vector masks allow operations to be applied selectively.

Conceptually:

```
Vector:
[10 20 30 40]

Mask:
[ T   F   T   F ]

Operation:
multiply by 2

Result:
[20 20 60 40]
```

Only the lanes enabled by the mask participate in the selected operation.

This is useful for conditional vectorized algorithms.

Typical Use Cases

Vector API is particularly useful for computational workloads such as:

- Image processing
 - Audio/video processing
 - Scientific computing
 - Financial calculations
 - Machine-learning algorithms
 - Signal processing
 - Cryptographic primitives
 - Numerical simulations
 - Large array transformations
 - Matrix/vector calculations
-

Compilation and Execution

Because this is an **incubator module**, it must be made available to the compiler/runtime.

For Java 26, a typical command-line invocation is:

```
javac --add-modules jdk.incubator.vector VectorAPI.java
```

Run:

```
java --add-modules jdk.incubator.vector VectorAPI
```

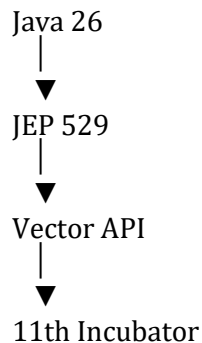
The incubator status means the API is still being developed and can change before eventually becoming a standard API.

Eleventh Incubator

The Vector API has undergone a long sequence of incubator iterations.

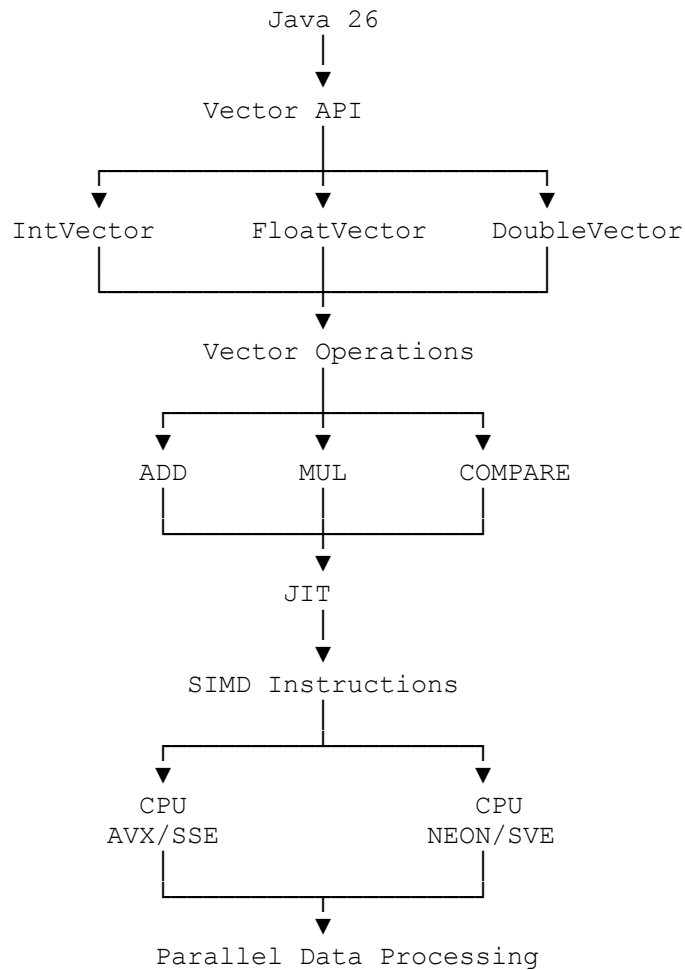
Java 16 → First Incubator
Java 17 → Second
Java 18 → Third
Java 19 → Fourth
Java 20 → Fifth
Java 21 → Sixth
Java 22 → Seventh
Java 23 → Eighth
Java 24 → Ninth
Java 25 → Tenth
Java 26 → Eleventh

Thus:

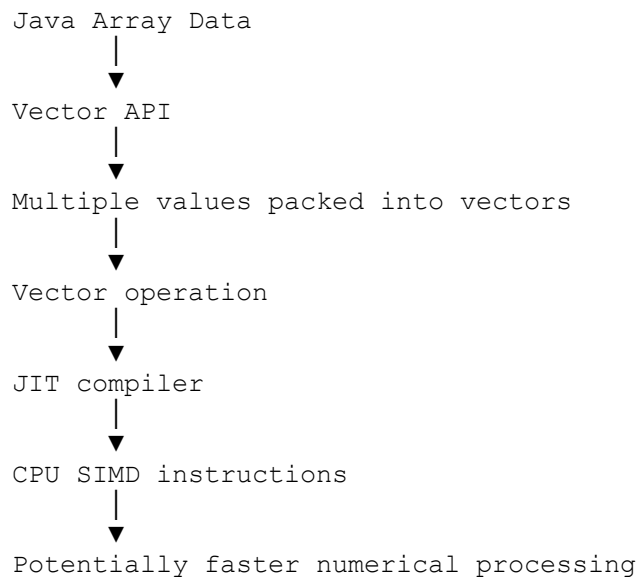


The continued incubation reflects the desire to refine the API based on implementation experience and developer feedback.

Complete Architecture



Key Takeaway



One-line definition

Java 26's Vector API (JEP 529, Eleventh Incubator) provides a portable Java API for expressing SIMD computations so that the JIT compiler can map vector operations to efficient hardware vector instructions, enabling multiple data elements to be processed in parallel.

Java 26 — Primitive Types in Patterns (Fourth Preview)

Java Version: 26

Year: 2026

Feature: Primitive Types in Patterns (Fourth Preview)

UniqueClassName: PrimitiveTypesPatterns

JEP: 530 — Primitive Types in Patterns, Fourth Preview

Definition

Primitive Types in Patterns extends Java's pattern matching capabilities so that **primitive values can participate directly in type patterns, switch patterns, and related pattern-matching constructs.**

Traditionally, Java pattern matching was primarily designed around reference types:

```
Object value = 42;
```

```
if (value instanceof Integer i) {  
    System.out.println(i);  
}
```

Java 26's fourth preview continues the effort to make pattern matching work naturally with **primitive types**, including:

```
byte  
short  
int  
long  
float  
double  
char  
boolean
```

The goal is to make primitive and reference pattern matching more uniform while providing well-defined rules for conversions, compatibility, and possible runtime failures.

Traditional Java vs Primitive Patterns

Traditional switch

Historically, primitive handling often required explicit cases or conversions:

```
int value = 10;
```

```
switch (value) {  
    case 0 -> System.out.println("Zero");  
    case 10 -> System.out.println("Ten");  
    default -> System.out.println("Other");  
}
```

Primitive type patterns allow the type itself to become part of the pattern:

```
Object value = Integer.valueOf(10);
```

```
switch (value) {  
    case int i -> System.out.println("int: " + i);  
    default -> System.out.println("Other");  
}
```

The pattern:

```
case int i
```

means, conceptually:

Does the value match int?



Extract the primitive value

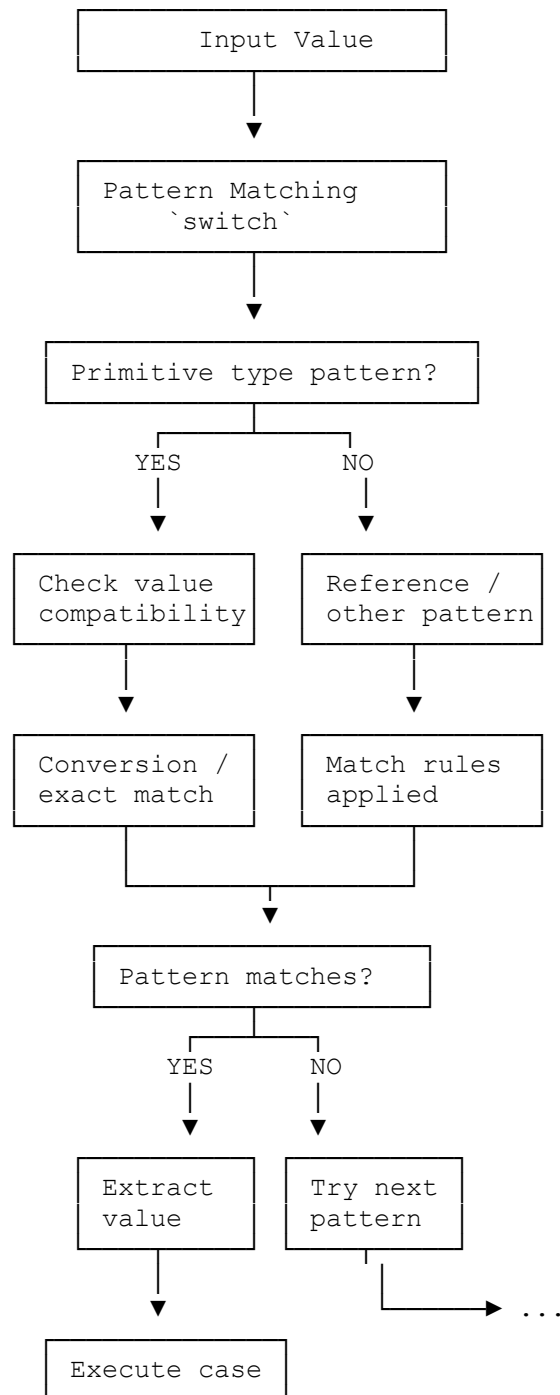


Store it in i

Simple Definition

Primitive Types in Patterns allows Java pattern matching to work directly with primitive types, enabling primitive type patterns and primitive-aware switch constructs while defining safe and predictable conversion rules.

Flowchart



Primitive Type Patterns

The feature allows primitive types to be used as patterns.

For example:

```
case int i
```

or:

```
case long l
```

or:

```
case double d
```

Conceptually:

```
Pattern
|
├─ int
├─ long
├─ float
├─ double
├─ byte
├─ short
├─ char
└─ boolean
```

Java Code Example

Because this is a **preview feature**, preview support must be enabled when compiling and running the code.

```
public class PrimitiveTypesPatterns {

    public static void main(String[] args) {

        Object value = Integer.valueOf(100);

        String result = switch (value) {

            case int i ->
                "Integer primitive value: " + i;

            case long l ->
                "Long primitive value: " + l;

            case double d ->
```

```

        "Double primitive value: " + d;

        default ->
            "Other value";
    };

    System.out.println(result);
}

```

Conceptually, the value:

`Integer.valueOf(100)`

is matched against the primitive pattern:

`case int i`

and the extracted value becomes:

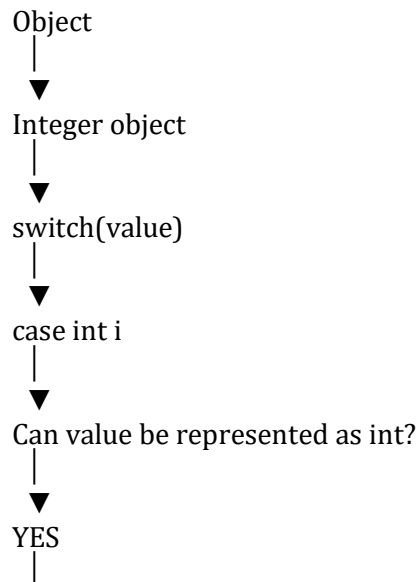
`i = 100`

Pattern Matching Flow

For:

`Object value = Integer.valueOf(100);`

the flow is:



▼
i = 100
|
▼
Execute case

Example with Multiple Primitive Types

```
public class PrimitiveTypesPatterns {  
    static String describe(Object value) {  
        return switch (value) {  
            case byte b ->  
                "byte: " + b;  
            case short s ->  
                "short: " + s;  
            case int i ->  
                "int: " + i;  
            case long l ->  
                "long: " + l;  
            case float f ->  
                "float: " + f;  
            case double d ->  
                "double: " + d;  
            case char c ->  
                "char: " + c;  
            case boolean b ->  
                "boolean: " + b;  
            default ->  
                "reference type: " + value;  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(  
            describe(Integer.valueOf(42))  
        );  
  
        System.out.println(  
            describe(Long.valueOf(42L))  
        );  
    }  
}
```

```

        System.out.println(
            describe(Double.valueOf(3.14))
        );

        System.out.println(
            describe("Java")
        );
    }
}

```

The conceptual result is:

Int	: 42
Long	: 42
Double	: 3.14
reference type	: Java

The exact applicability of each pattern depends on Java's primitive-pattern conversion rules.

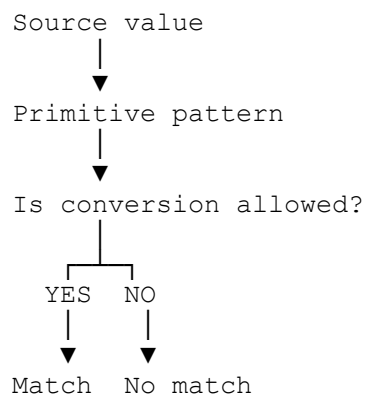
Primitive Conversion

One of the important aspects of this JEP is that primitive patterns are not simply equivalent to:

`instanceof`

Java must determine whether a value can be converted to the primitive type represented by the pattern.

For example:



This is particularly important when patterns involve different primitive widths.

Example — int and long

```
public class PrimitiveTypesPatterns {  
  
    public static void main(String[] args) {  
  
        Object value = Long.valueOf(100L);  
  
        String result = switch (value) {  
  
            case int i ->  
                "int: " + i;  
  
            case long l ->  
                "long: " + l;  
  
            default ->  
                "other";  
        };  
  
        System.out.println(result);  
    }  
}
```

Here Java's pattern-matching rules determine whether the Long value can match the int pattern or should proceed to the long pattern.

The important principle is:

Pattern matching must not silently produce an incorrect primitive value through an unsafe narrowing conversion.

Primitive Patterns and switch

The feature becomes especially useful when combined with modern switch pattern matching.

```
static String classify(Object value) {  
  
    return switch (value) {  
  
        case int i when i > 0 ->  
            "Positive integer";  
  
        case int i ->  
            "Zero or negative integer";  
  
        case long l ->
```

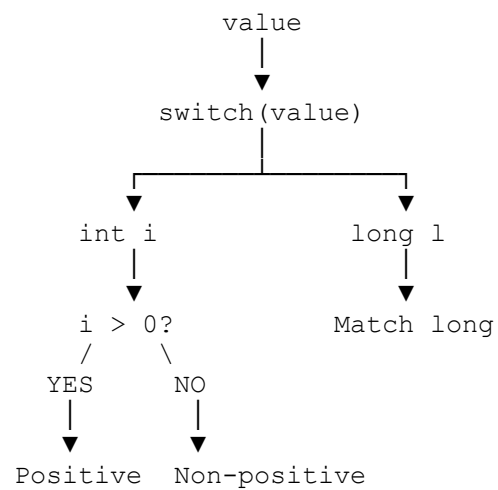
```

        "Long value: " + l;

    default ->
        "Other";
    };
}

```

Conceptually:



Primitive Patterns vs Traditional Boxing

Before primitive type patterns, developers often encountered boxing:

```

Object value = Integer.valueOf(42);

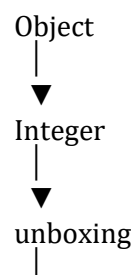
if (value instanceof Integer integer) {

    int number = integer;

    System.out.println(number);
}

```

There are two conceptual stages:



▼
int

Primitive patterns aim to make the pattern itself express the desired primitive type:

```
if (value instanceof int i) {  
    System.out.println(i);  
}
```

The exact pattern contexts and rules are governed by the JEP's preview specification.

Example — Numeric Classification

```
public class PrimitiveTypesPatterns {  
    static String classify(Number number) {  
        return switch (number) {  
            case Integer i ->  
                "Integer object: " + i;  
            case Long l ->  
                "Long object: " + l;  
            case Double d ->  
                "Double object: " + d;  
            default ->  
                "Other Number";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(  
            classify(Integer.valueOf(10))  
        );  
  
        System.out.println(  
            classify(Long.valueOf(20L))  
        );  
  
        System.out.println(  
            classify(Double.valueOf(30.5))  
        );  
    }  
}
```

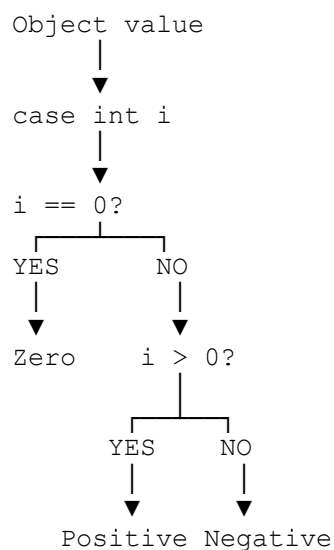
This example uses **reference patterns**, whereas the new feature allows primitive patterns to participate directly in pattern matching.

Primitive Pattern + Guard

Primitive patterns become particularly expressive when combined with guards:

```
static String classify(Object value) {  
    return switch (value) {  
        case int i when i == 0 ->  
            "Zero";  
        case int i when i > 0 ->  
            "Positive";  
        case int i ->  
            "Negative";  
        default ->  
            "Not an integer";  
    };  
}
```

Flow:



Why Is This Useful?

1. Less Boxing-Oriented Code

Primitive values can participate more naturally in pattern matching.

2. More Expressive switch

Primitive types can be incorporated into sophisticated switch patterns.

3. Better Type Uniformity

Java can increasingly use a common pattern-matching model for:

Reference types

+

Primitive types

4. Better Data-Oriented Programming

Pattern matching becomes more useful for algorithms that operate heavily on numeric and primitive data.

Important Safety Consideration

Primitive conversions can lose information.

For example:

long

|



int

|



Possible information loss

Therefore, primitive pattern matching needs carefully defined rules to avoid surprising results.

A conceptual safe matching process is:

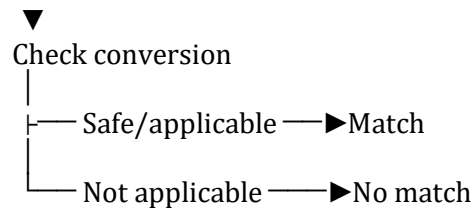
Value

|



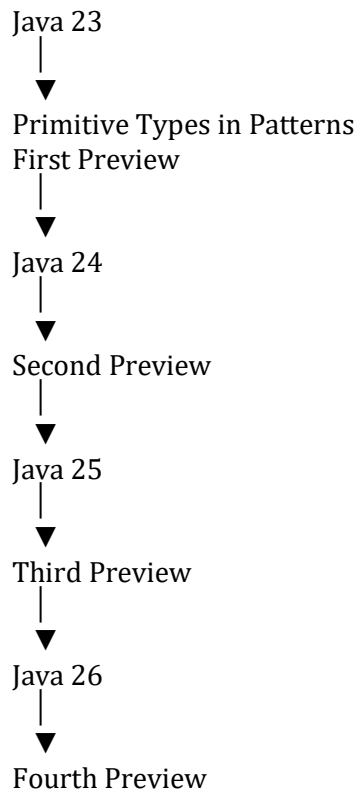
Candidate primitive pattern

|



Fourth Preview

The feature has evolved through several Java releases:



So Java 26 continues refining the feature rather than making it a final permanent language feature.

Because it is a preview feature, its syntax and semantics may still change in a future Java release.

Compilation

Since this is a preview language feature, use preview compilation.

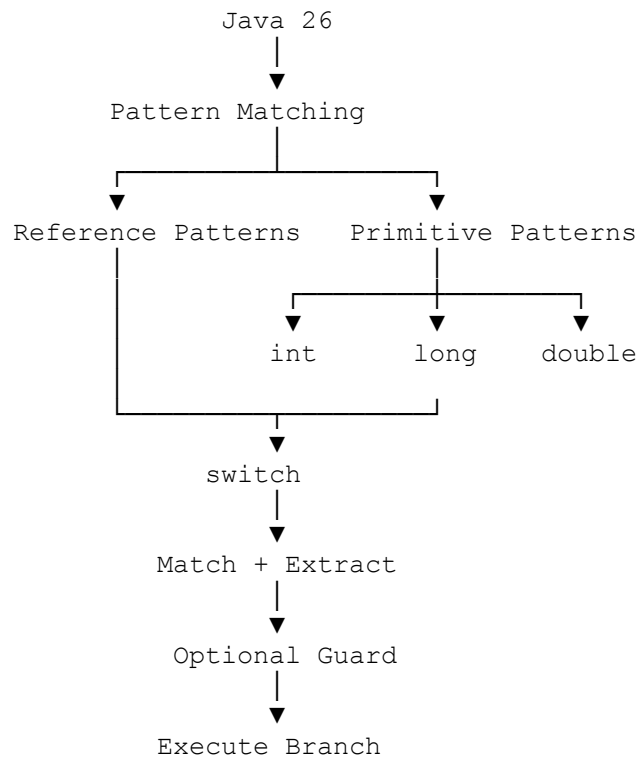
Conceptually:

```
javac --enable-preview --release 26 PrimitiveTypesPatterns.java
```

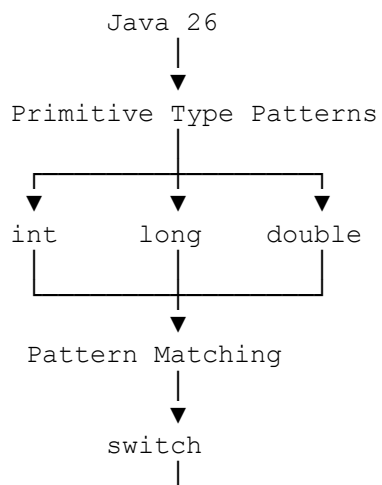
Run with:

```
java --enable-preview PrimitiveTypesPatterns
```

Complete Architecture



Key Takeaway



▼
Match + Extract Value

One-line definition

Java 26's Primitive Types in Patterns (JEP 530, Fourth Preview) extends pattern matching to primitive types, allowing primitive-aware patterns to be used in constructs such as switch and enabling more expressive type-based processing of numeric and other primitive values while defining precise rules for primitive conversions.